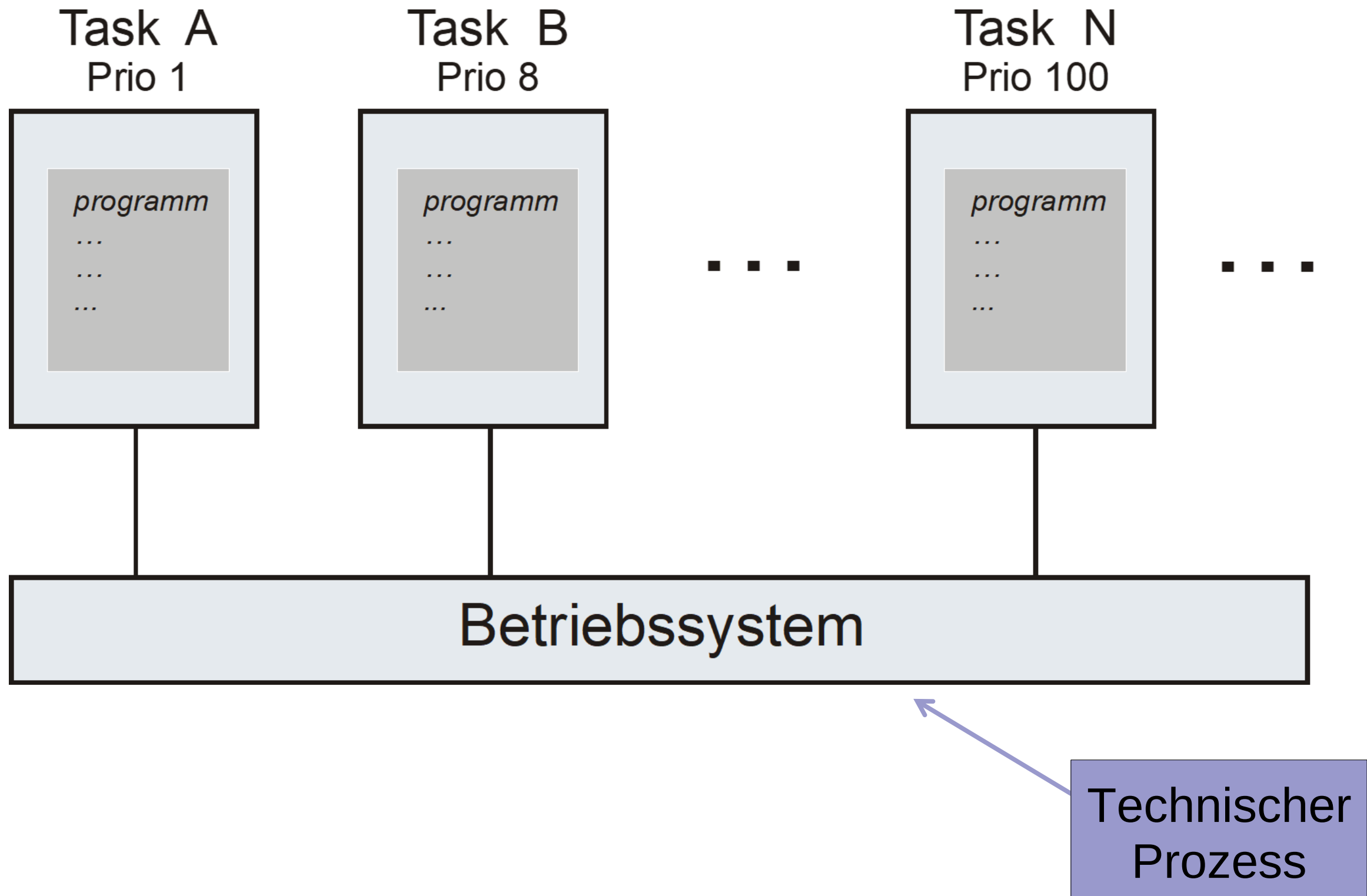
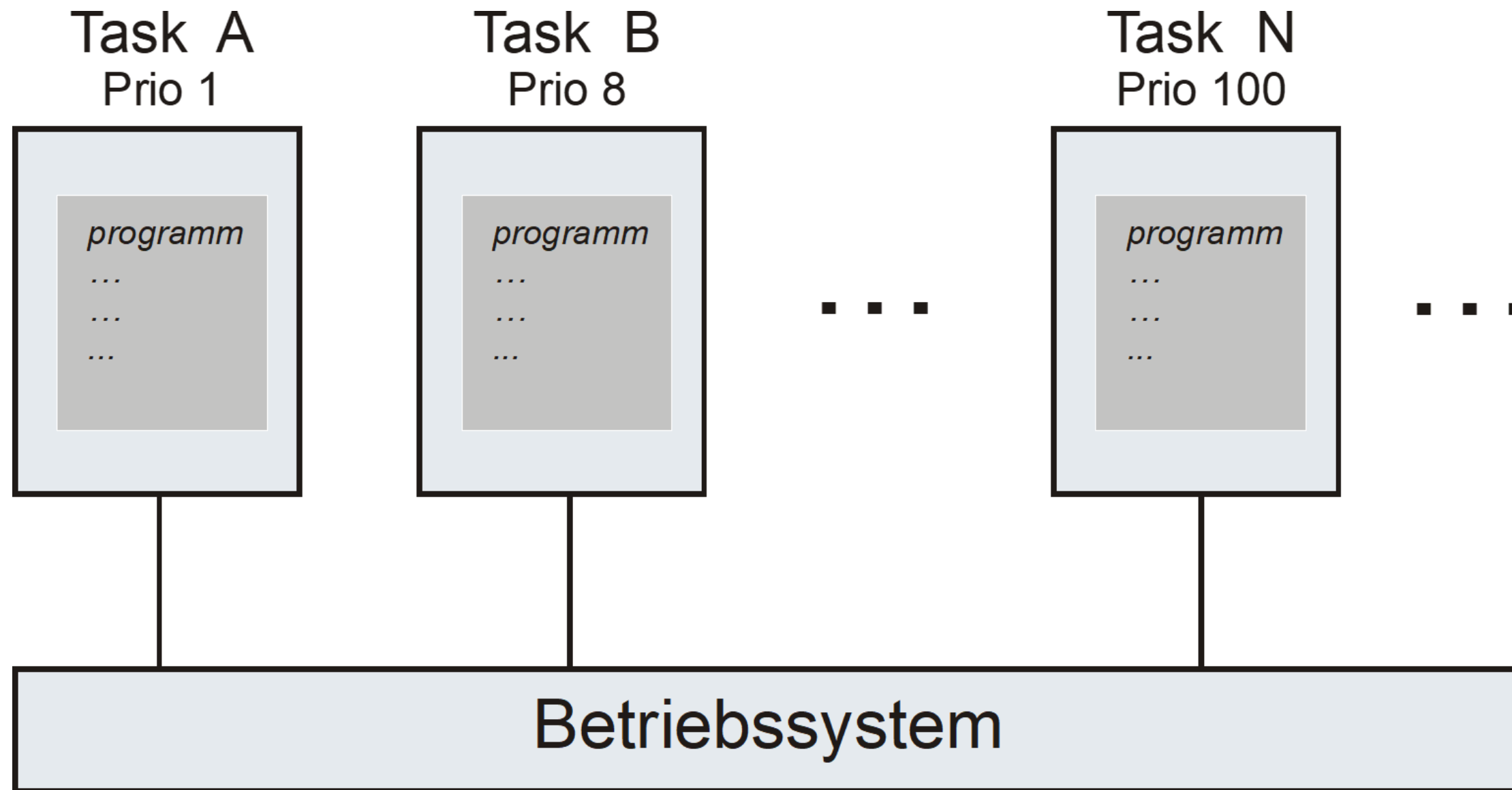

Ereignis-basierte Einplanung von Tasks

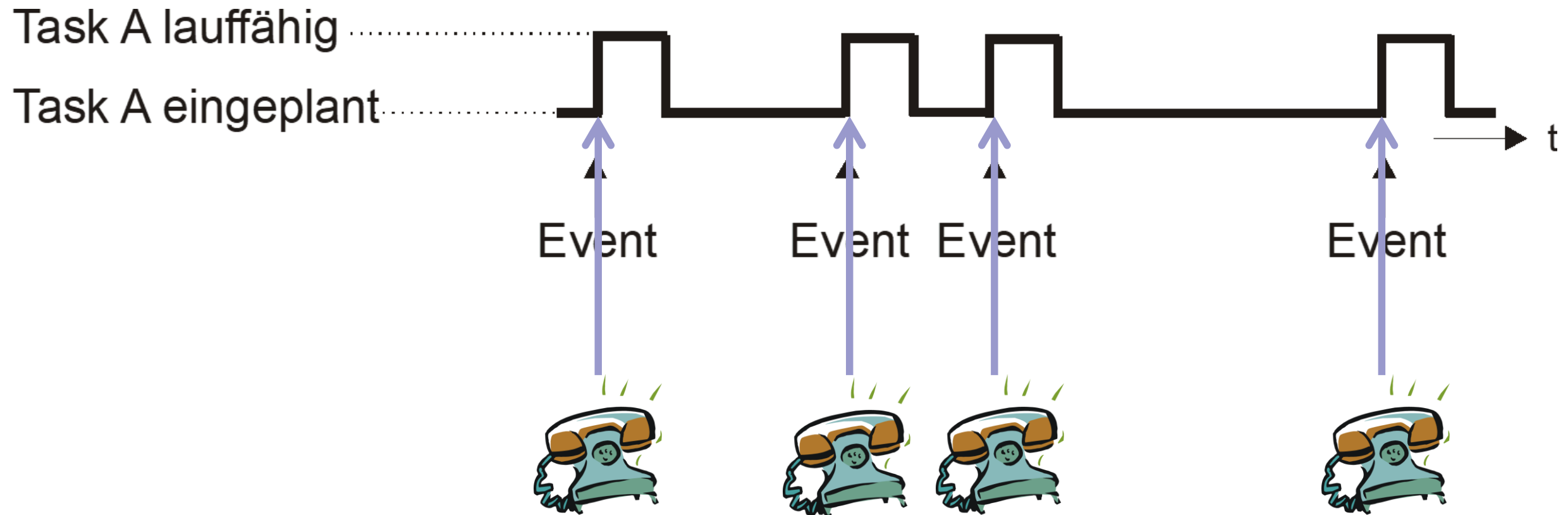
Ereigniseinplanung von Tasks



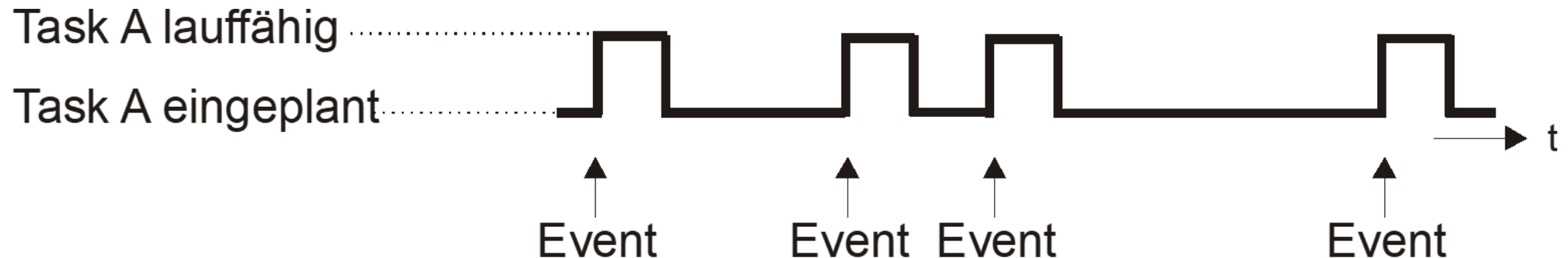
Ereigniseinplanung von Tasks



Ereigniseinplanung von Tasks



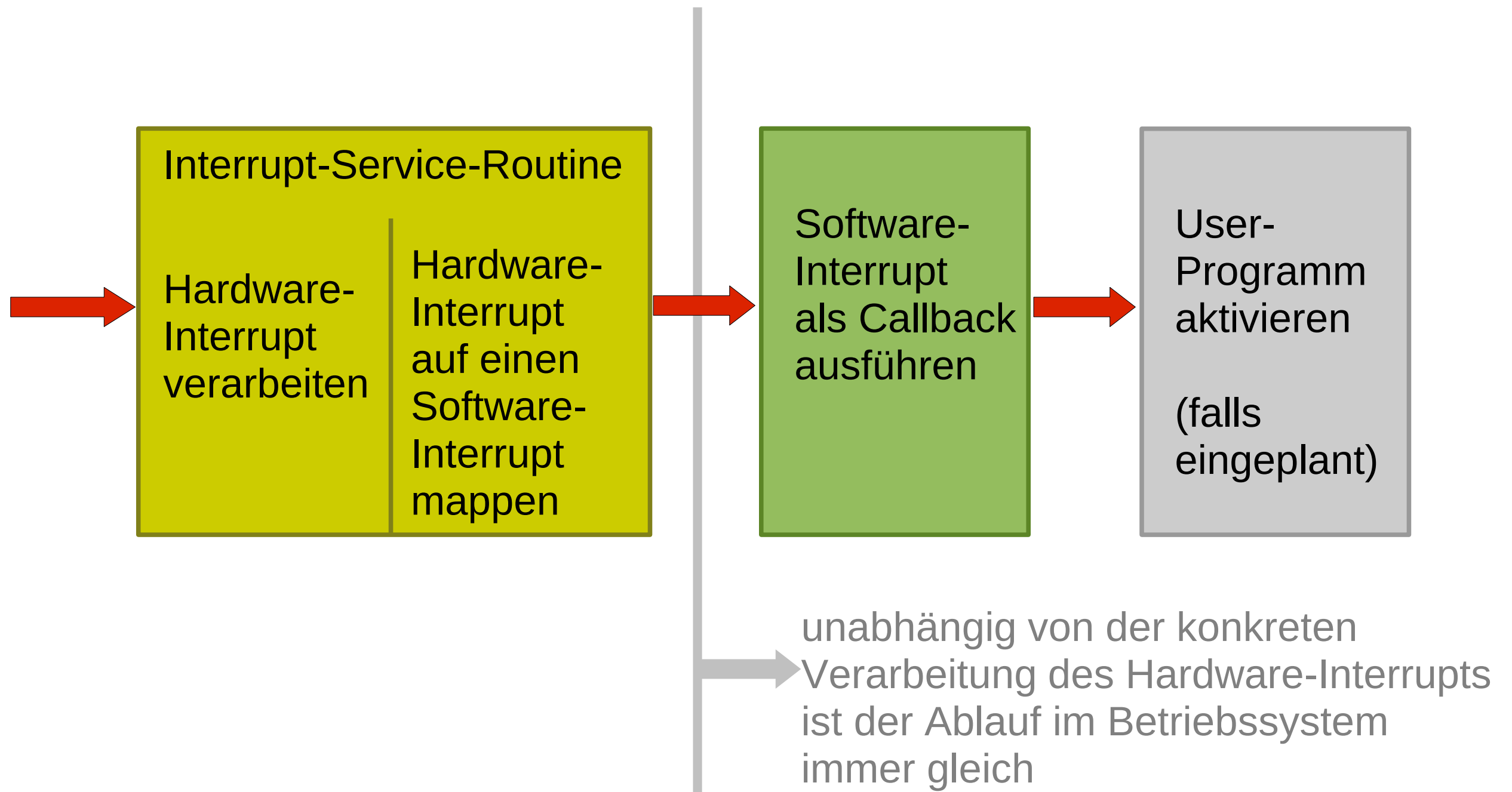
Ereigniseinplanung von Tasks



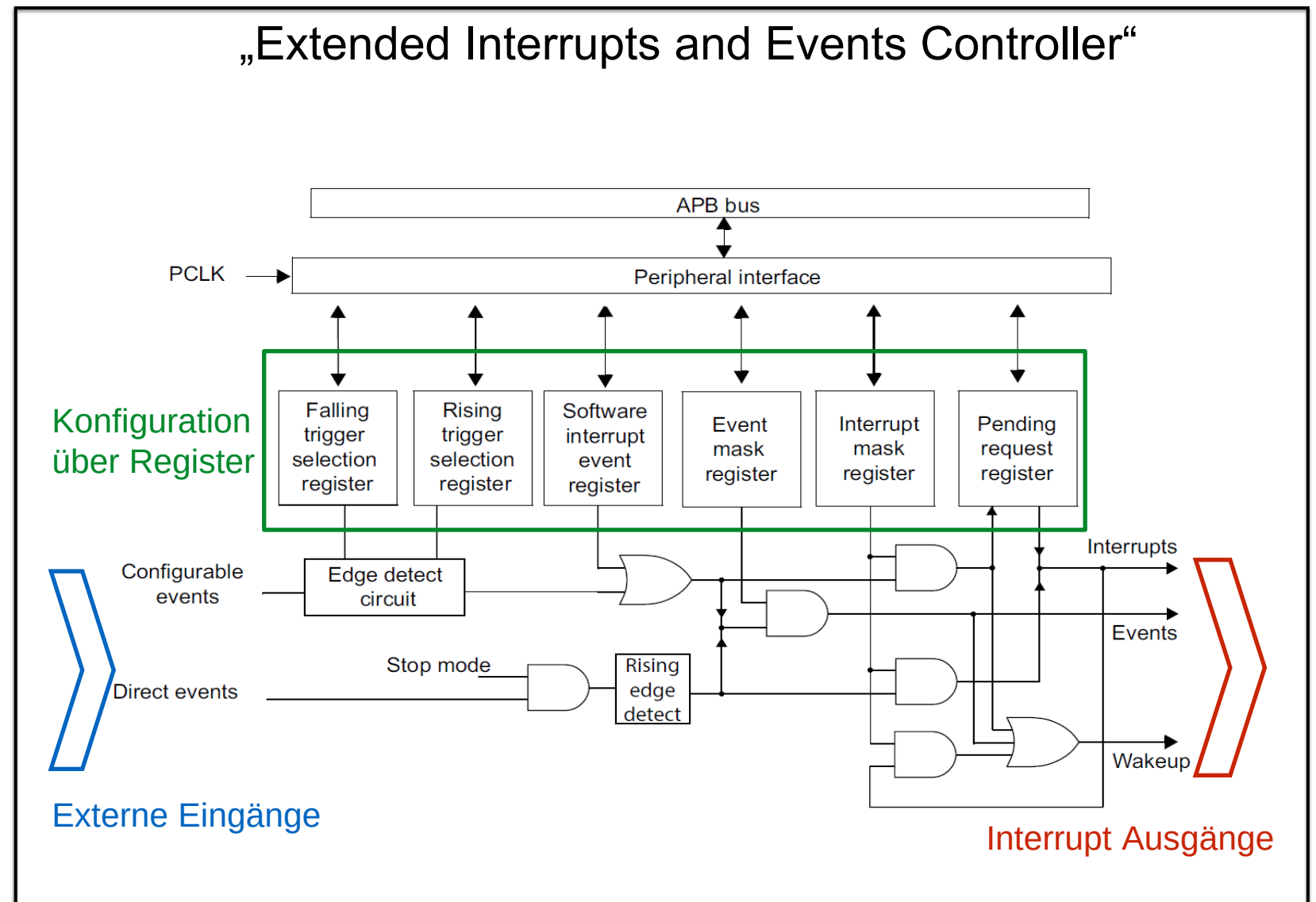
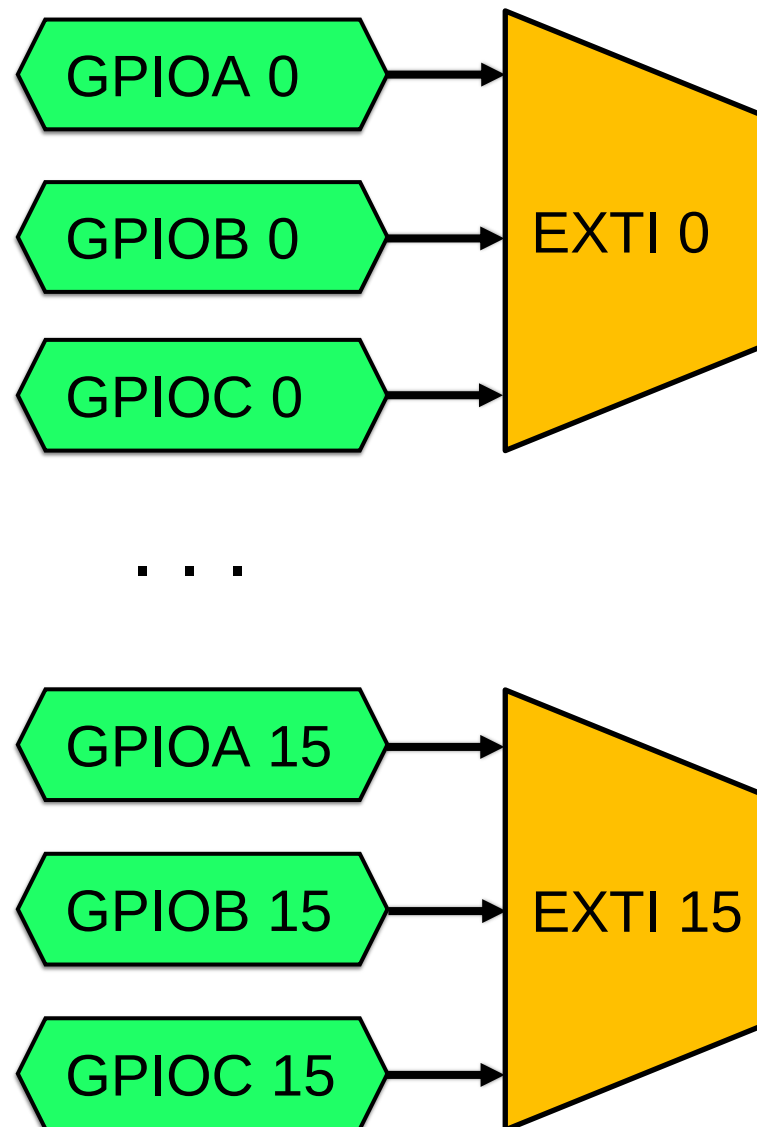
Umsetzung in der STM32-IDE durch definieren der Funktion:

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){  
    switch(GPIO_Pin):  
        case 1 << 0:  
            Callback GPIO 0  
        case 1 << 1:  
            Callback GPIO 1  
        ...  
    return;  
}
```

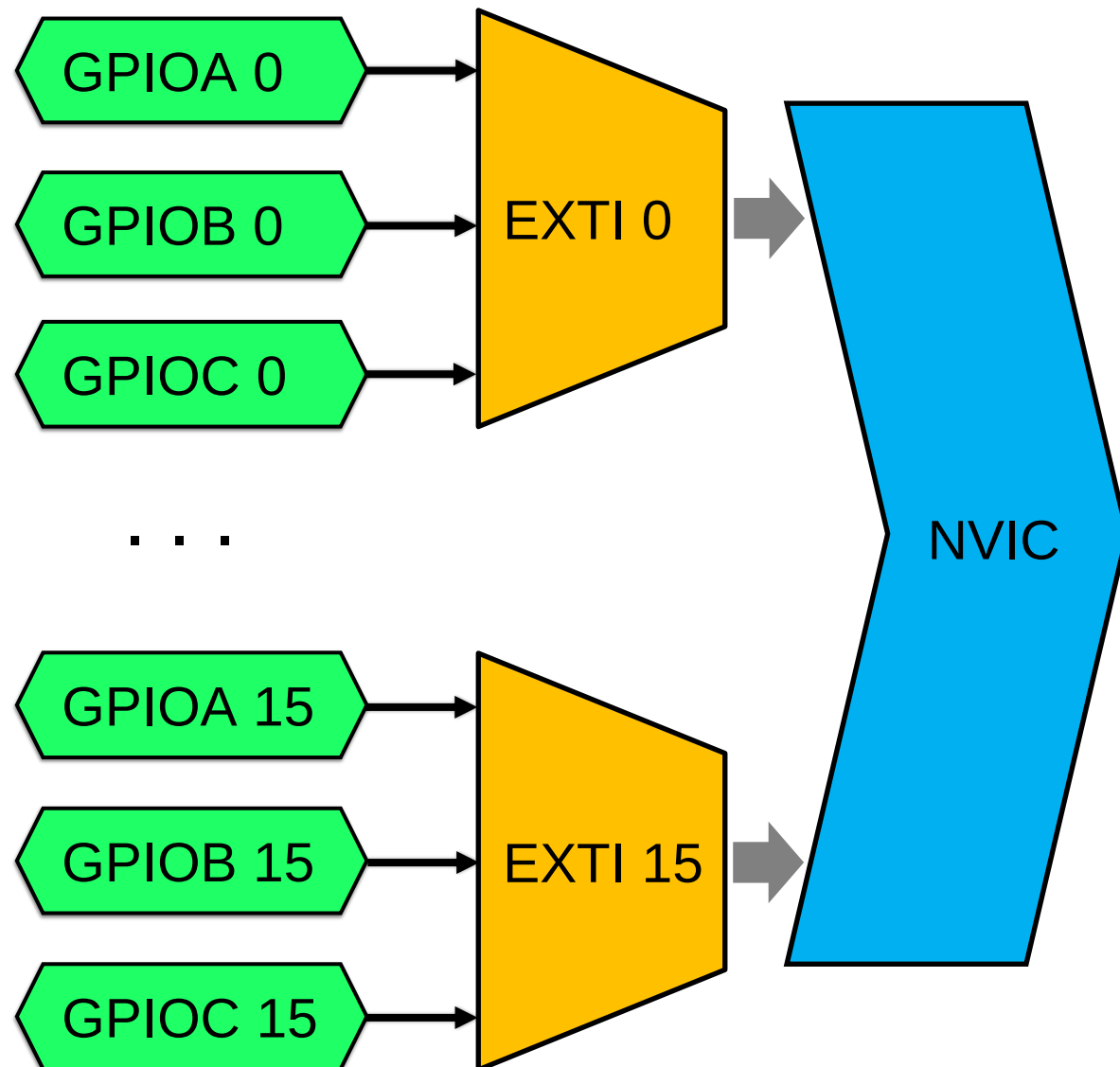
Ereigniseinplanung von Tasks



Ereigniseinplanung von Tasks



Ereigniseinplanung von Tasks



„Nested Vectored Interrupt Controller“

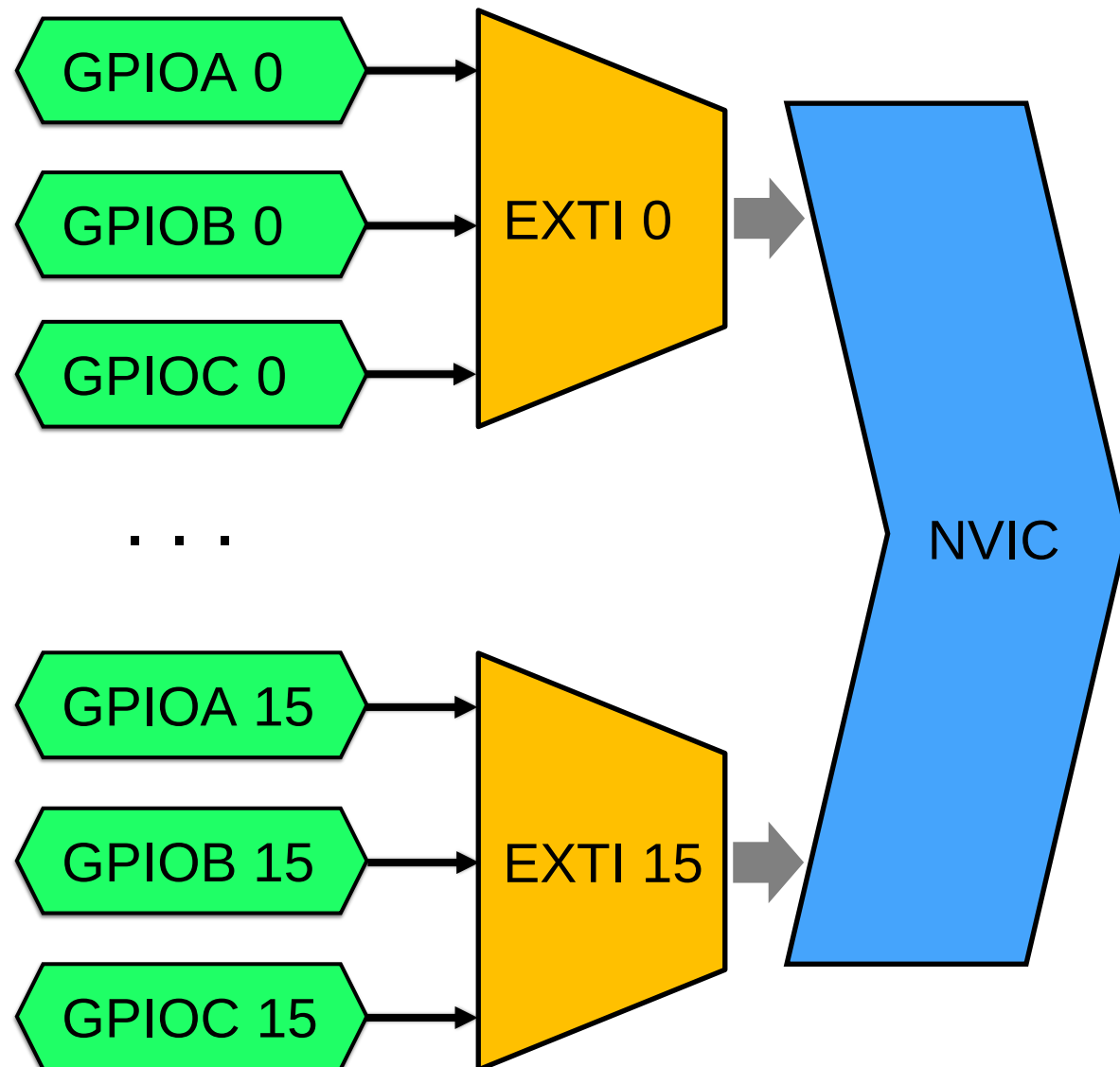
Zuständig für Management und Priorisierung der verschiedenen Interrupt-Quellen (für unser Board ~90)

Position	Priority	Type of priority	Acronym	Description	Address
76	83	settable	SWPMI1	SWPMI1 global interrupt	0x0000 0170
77	84	settable	TSC	TSC global interrupt	0x0000 0174
78	85	settable	LCD ⁽²⁾		
79	86	settable	AES ⁽³⁾		
80	87	settable	RNG ⁽⁴⁾		
81	88	settable	FPU		
82	89	settable	HASH		
83	90	settable	I2C4_I		
84	91	settable	I2C4_I		
85	92	settable	DCMI		
86	93	settable	CAN2		
87	94	settable	CAN2		
88	95	settable	CAN2		
89	96	settable	CAN2		
90	97	settable	DMA2		

Position	Priority	Type of priority	Acronym	Description	Address
12	19	settable	DMA1_CH2	DMA1 channel 2 interrupt	0x0000 0070
13	20	settable	DMA1_CH3		
14	21	settable	DMA1_CH4		
15	22	settable	DMA1_CH5		
16	23	settable	DMA1_CH6		
17	24	settable	DMA1_CH7		
18	25	settable	ADC1_2		
19	26	settable	CAN1_TX		
20	27	settable	CAN1_RX0		
21	28	settable	CAN1_RX1		
22	29	settable	CAN1_SCE		
23	30	settable	EXTI9_5		
24	31	settable	TIM1_BRK/TIM1		
25	32	settable	TIM1_UP/TIM1		
26	33	settable	TIM1_TRG_CC/TIM17		
27	34	settable	TIM1_CC		
28	35	settable	TIM2		
29	36	settable	TIM3		
30	37	settable	TIM4		
31	38	settable	I2C1_EV		
32	39	settable	I2C1_ER		
33	40	settable	I2C2_EV		
34	41	settable	I2C2_ER		
35	42	settable	SPI1		
36	43	settable	SPI2		
37	44	settable	USART1		
38	45	settable	USART2		
39	46	settable	USART3		
40	47	settable	EXTI15_10		
41	48	settable	RTC_ALARM		
42	49	settable	DFSDM1_FLT3		
43	50	settable	TIM8_BRK		
44	51	settable	TIM8_UP	TIM8 Update interrupt	0x0000 00F0
45	52	settable	TIM8_TRG_COM	TIM8 trigger and commutation interrupt	0x0000 00F4
46	53	settable	TIM8_CC		
47	54	settable	ADC3		
48	55	settable	FMC		
49	56	settable	SDMMC1		
50	57	settable	TIM5		
51	58	settable	SPI3		
52	59	settable	UART4		
53	60	settable	UART5		
54	61	settable	TIM6_DAC/UNT		
55	62	settable	TIM7		
56	63	settable	DMA2_CH1		
57	64	settable	DMA2_CH2		
58	65	settable	DMA2_CH3		
59	66	settable	DMA2_CH4		
60	67	settable	DMA2_CH5		
61	68	settable	DFSDM1_FLT1		
62	69	settable	DFSDM1_FLT2		
63	70	settable	DFSDM1_FLT3		
64	71	settable	COMP		
65	72	settable	LPTIM1		
66	73	settable	LPTIM2		
67	74	settable	OTG_FS ⁽¹⁾		
68	75	settable	DMA2_CH6		
69	76	settable	DMA2_CH7		
70	77	settable	LPUART1		
71	78	settable	QUADSPI		
72	79	settable	I2C3_EV		
73	80	settable	I2C3_ER		
74	81	settable	SAI1		
75	82	settable	SAI2		

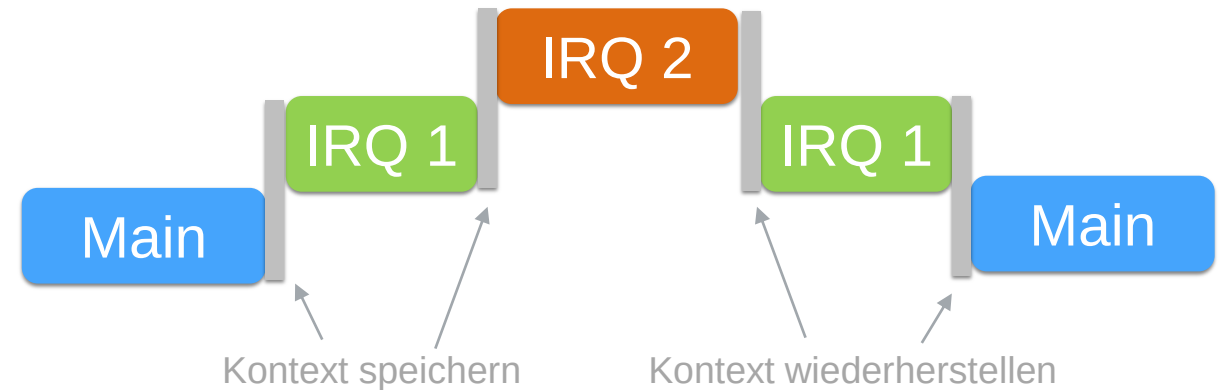
Position	Priority	Type of priority	Acronym	Description	Address
-	-	-	-	Reserved	0x0000 0000
-	-3	fixed	Reset	Reset	0x0000 0004
-	-2	fixed	NMI	Non maskable interrupt. The RCC Clock Security System (CSS) is linked to the NMI vector.	0x0000 0008
-	-1	fixed	HardFault	All classes of fault	0x0000 000C
-	0	settable	MemManage	Memory management	0x0000 0010
-	1	settable	BusFault	Pre-fetch fault, memory access fault	0x0000 0014
-	2	settable	UsageFault	Undefined instruction or illegal state	0x0000 0018
-	-	-	-	Reserved	0x0000 001C - 0x0000 0028
-	3	settable	SVCall	System service call via SWI instruction	0x0000 002C
-	4	settable	Debug	Monitor	0x0000 0030
-	-	-	-	Reserved	0x0000 0034
-	5	settable	PendSV	Pendable request for system service	0x0000 0038
-	6	settable	SysTick	System tick timer	0x0000 003C
0	7	settable	WWDG	Window Watchdog interrupt	0x0000 0040
1	8	settable	PVD_PVM	PVD/PVM1/PVM2/PVM3/PVM4 through EXTI lines 16/35/36/37/38 interrupts	0x0000 0044
2	9	settable	RTC_TAMP_STAMP/CSSE	RTC Tamper or TimeStamp /CSS on LSE through EXTI line 19 interrupts	0x0000 0048
3	10	settable	RTC_WKUP	RTC Wakeup timer through EXTI line 20 interrupt	0x0000 004C
4	11	settable	FLASH	Flash global interrupt	0x0000 0050
5	12	settable	RCC	RCC global interrupt	0x0000 0054
6	13	settable	EXTI0	EXTI Line0 interrupt	0x0000 0058
7	14	settable	EXTI1	EXTI Line1 interrupt	0x0000 005C
8	15	settable	EXTI2	EXTI Line2 interrupt	0x0000 0060
9	16	settable	EXTI3	EXTI Line3 interrupt	0x0000 0064
10	17	settable	EXTI4	EXTI Line4 interrupt	0x0000 0068
11	18	settable	DMA1_CH1	DMA1 channel 1 interrupt	0x0000 006C

Ereigniseinplanung von Tasks

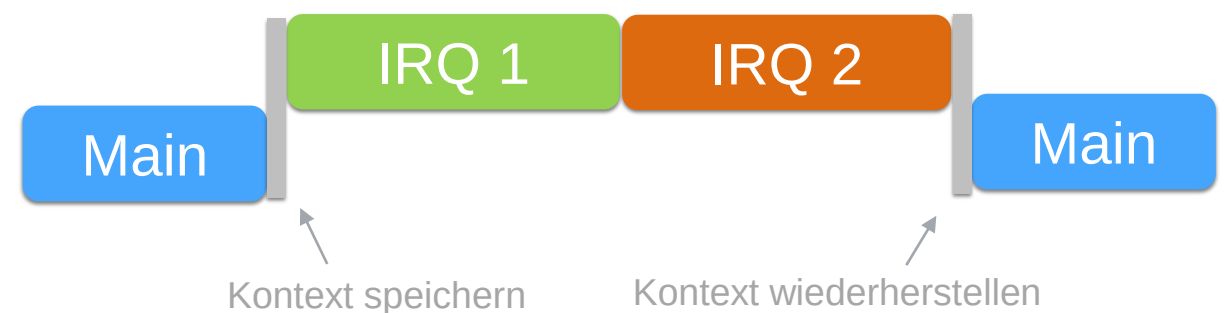


„Nested Vectored Interrupt Controller“

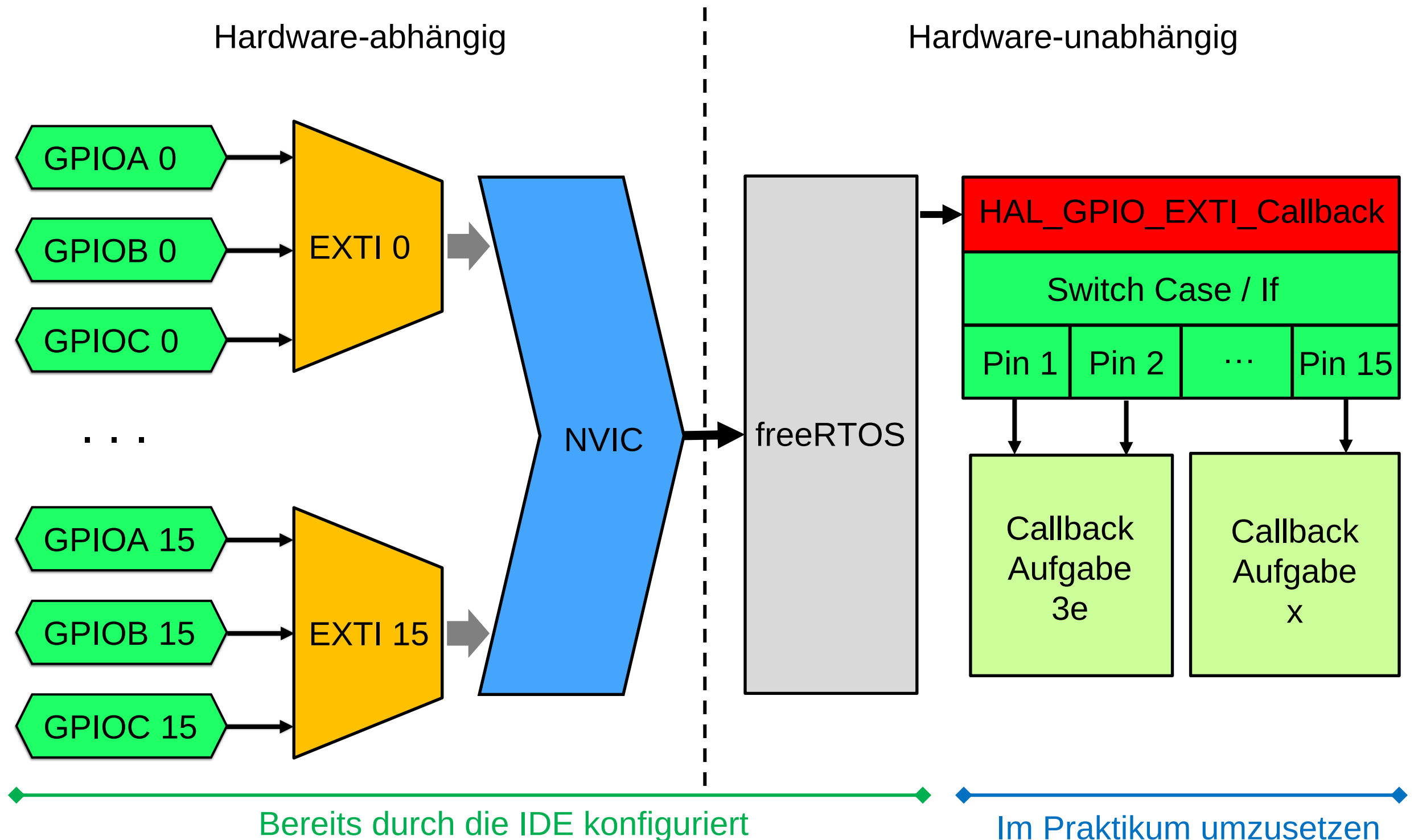
Auch Interrupts können von höher priorisierten Interrupts unterbrochen werden



Gleiche oder niedriger priorisierte Interrupts werden eingereiht



Ereigniseinplanung von Tasks



Zuordnung der EXTI Kanäle

Port	uint16_t Maske		HAL ID	Hardware	Aufgabe
EXTI 0	1	1<<0	GPIO A Pin 1	Drehgeber Nucleo Kanal A	Aufgabe 3e
EXTI 1	2	1<<1	GPIO A Pin 2	Drehgeber Nucleo Kanal B	
EXTI 2	4	1<<2	-	-	-
EXTI 3	8	1<<3	-	-	-
EXTI 4	16	1<<4	HAL GPIO B Pin 5	Endstop 1 Linearachse	Aufgabe 3f
EXTI 5	32	1<<5	HAL GPIO B Pin 6	Endstop 2 Linearachse	
EXTI 6	64	1<<6	HAL GPIO A Pin 7	Encoder Kanal A Linearachse	
EXTI 7	128	1<<7	HAL GPIO A Pin 8	Encoder Kanal B Linearachse	
EXTI 8	256	1<<8	HAL GPIO B Pin 9	Schalter 1	Aufgabe 5
EXTI 9	512	1<<9	HAL GPIO B Pin 10	Schalter 2	
EXTI 10	1024	1<<10	HAL GPIO B Pin 12	Schalter 3	
EXTI 11	2048	1<<11	-	-	-
EXTI 12	4096	1<<12	-	-	-
EXTI 13	8192	1<<13	-	-	-
EXTI 14	16.384	1<<14	-	-	-
EXTI 15	32768	1<<15	HAL GPIO A Pin 16	Tachometer PWM-Lüfter	Aufgabe 6

Beispiel Externer Interrupt

main.c

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
    switch(GPIO_Pin){
        case 1 << 0:
        case 1 << 1:
            ISR_3e();
            return;
        case 1 << 3:
            ISR_x();
        case 1 << 4:
        case 1 << 5:
        case 1 << 6:
        case 1 << 7:
            ISR_3f();
            return;
        case 1 << 8:
        case 1 << 9:
        case 1 << 10:
            ISR_5();
            return;
        case 1 << 15:
            ISR_6a();
            return;
    }
    return;
}

int main(void){
    ...
    TaskHandle_t Handle = NULL;
    xTaskCreate(start_x, "Aufgabe X", 128, NULL, 10, &Handle);
    ...
    osKernelStart();
    while (1){}
}
```

Event auf EXTI3
(HAL GPIO PIN 4)

Startet Funktion
ISR_x()

aufgabe_x.c

```
static TaskHandle_t ausgabe_Handle = NULL;

int count = 0;

void ISR_x(){
    count ++;
    return;
}

static void ausgabe(){
    for(;;){
        printf("Count: %d \r", (int)count);
        vTaskDelay(100);
    }
}

void start_x(){
    printf("Aufgabe X\n\n");
    xTaskCreate(ausgabe, "", 128, NULL, 10, &ausgabe_Handle);
    vTaskDelete(NULL);
}
```

Einfache Funktion, die
nie blockieren kann

Beispiel Externer Interrupt

main.c

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
    switch(GPIO_Pin){
        case 1 << 0:
        case 1 << 1:
            ISR_3e();
            return;
        case 1 << 3:
            ISR_x();
        case 1 << 4:
        case 1 << 5:
        case 1 << 6:
        case 1 << 7:
            ISR_3f();
            return;
        case 1 << 8:
        case 1 << 9:
        case 1 << 10:
            ISR_5();
            return;
        case 1 << 15:
            ISR_6a();
            return;
    }
    return;
}

int main(void){
    ...
    TaskHandle_t Handle = NULL;
    xTaskCreate(start_x, "Aufgabe X", 128, NULL, 10, &Handle);
    ...
    osKernelStart();
    while (1){}
}
```

Event auf EXTI3
(HAL GPIO PIN 4)

Startet Funktion
ISR_x()

aufgabe_x.c

```
static TaskHandle_t ausgabe_Handle = NULL;
static TaskHandle_t complex_ISR_Handle = NULL;

int count = 0;

void ISR_x(){
    xTaskResumeFromISR(complex_ISR_Handle);
    return;
}

static void complex_ISR(){
    for(;;){
        vTaskSuspend(NULL);
        count ++;
        ... // Längere Operationen
        printf("Count: %d \r", count);
    }
}

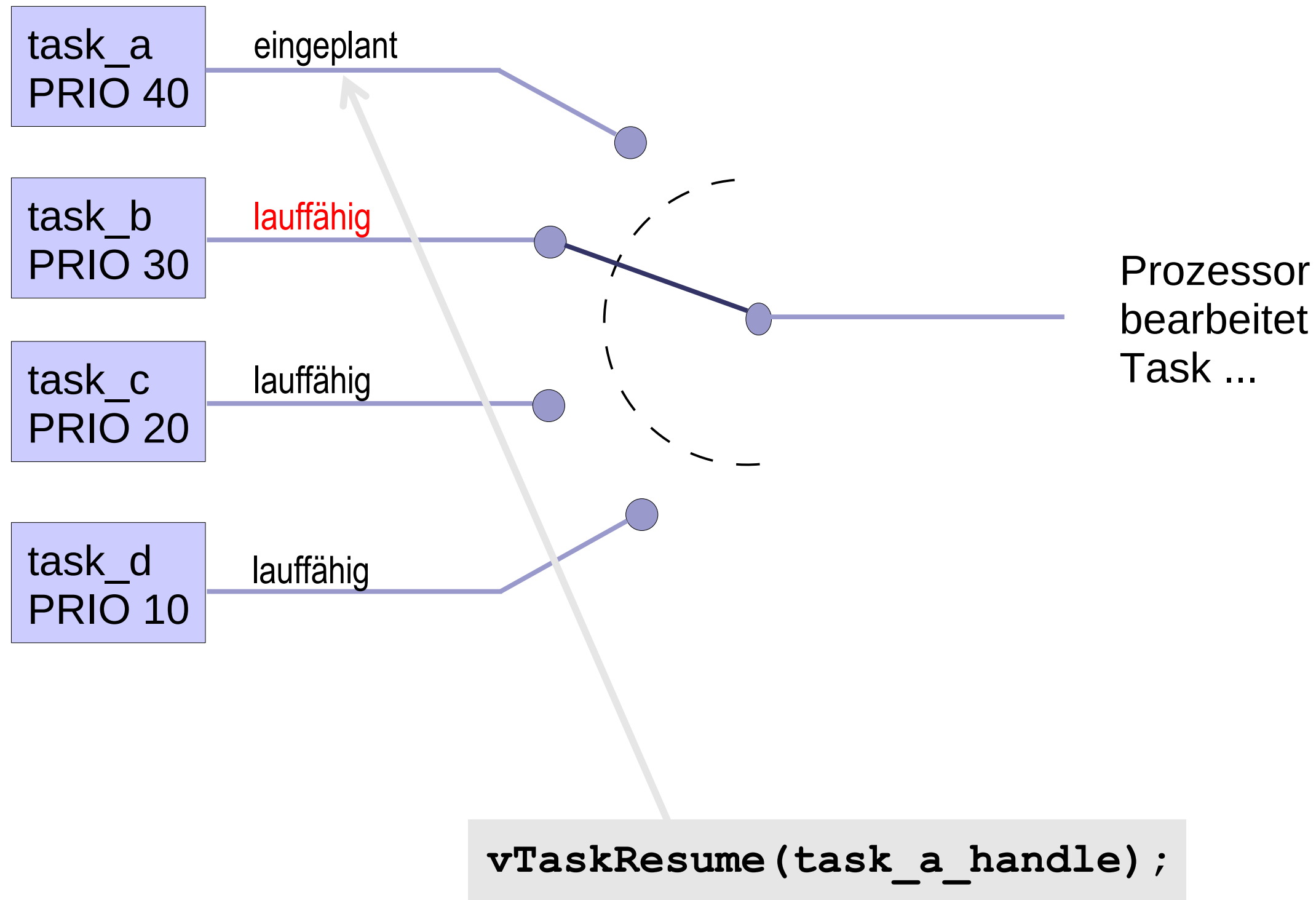
void start_x(){
    printf("Aufgabe X\n\n");
    xTaskCreate(ausgabe, "", 128, NULL, 10, &ausgabe_Handle);
    vTaskDelete(NULL);
}
```

Die Komplexere Operation wird von ISR_x() nur aktiviert.

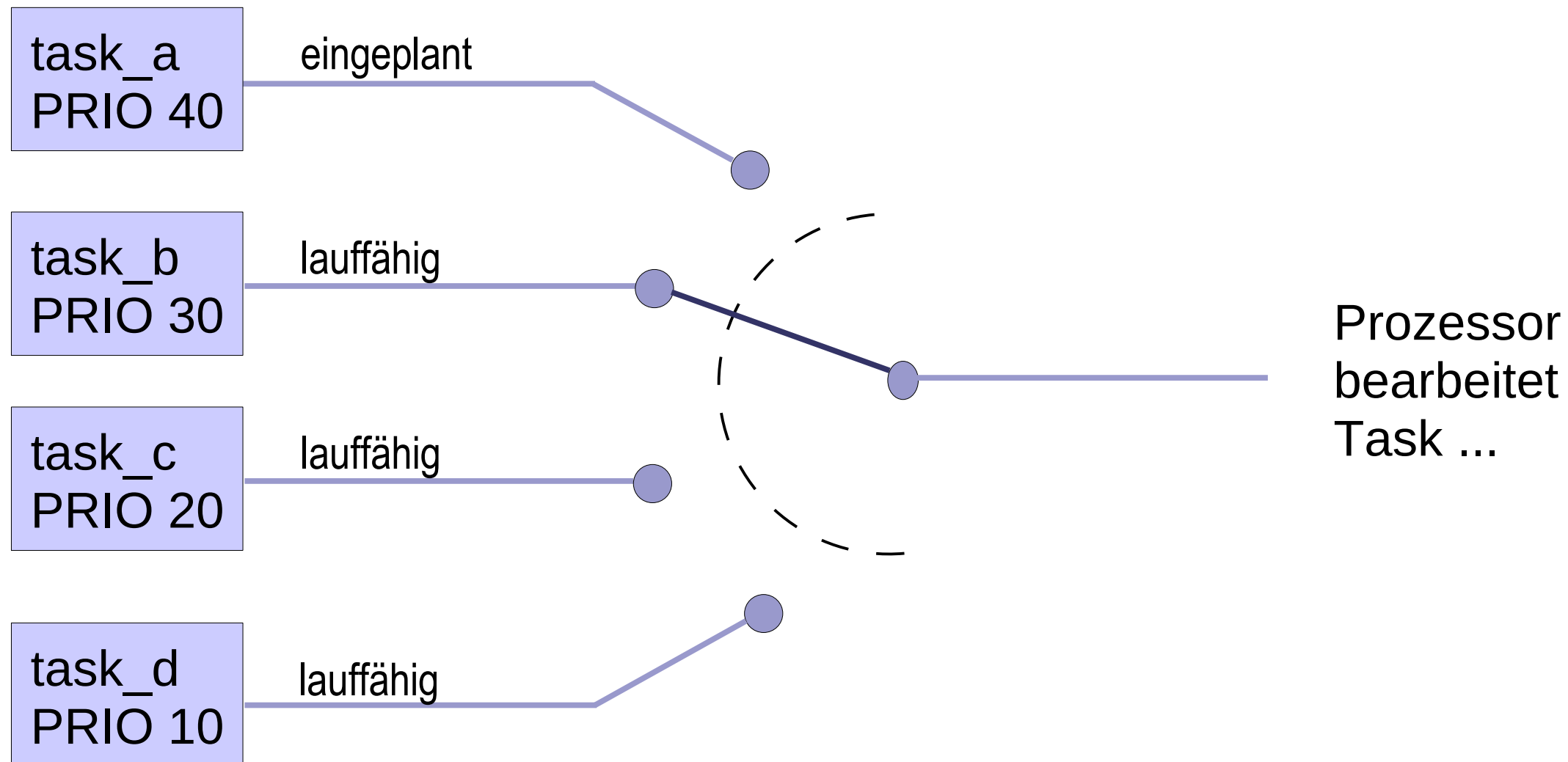
In einem vom NVIC ausgelösten Interrupt muss statt vTaskResume() die Funktion xTaskResumeFromISR() genutzt werden.

Einplanung im Multitasking

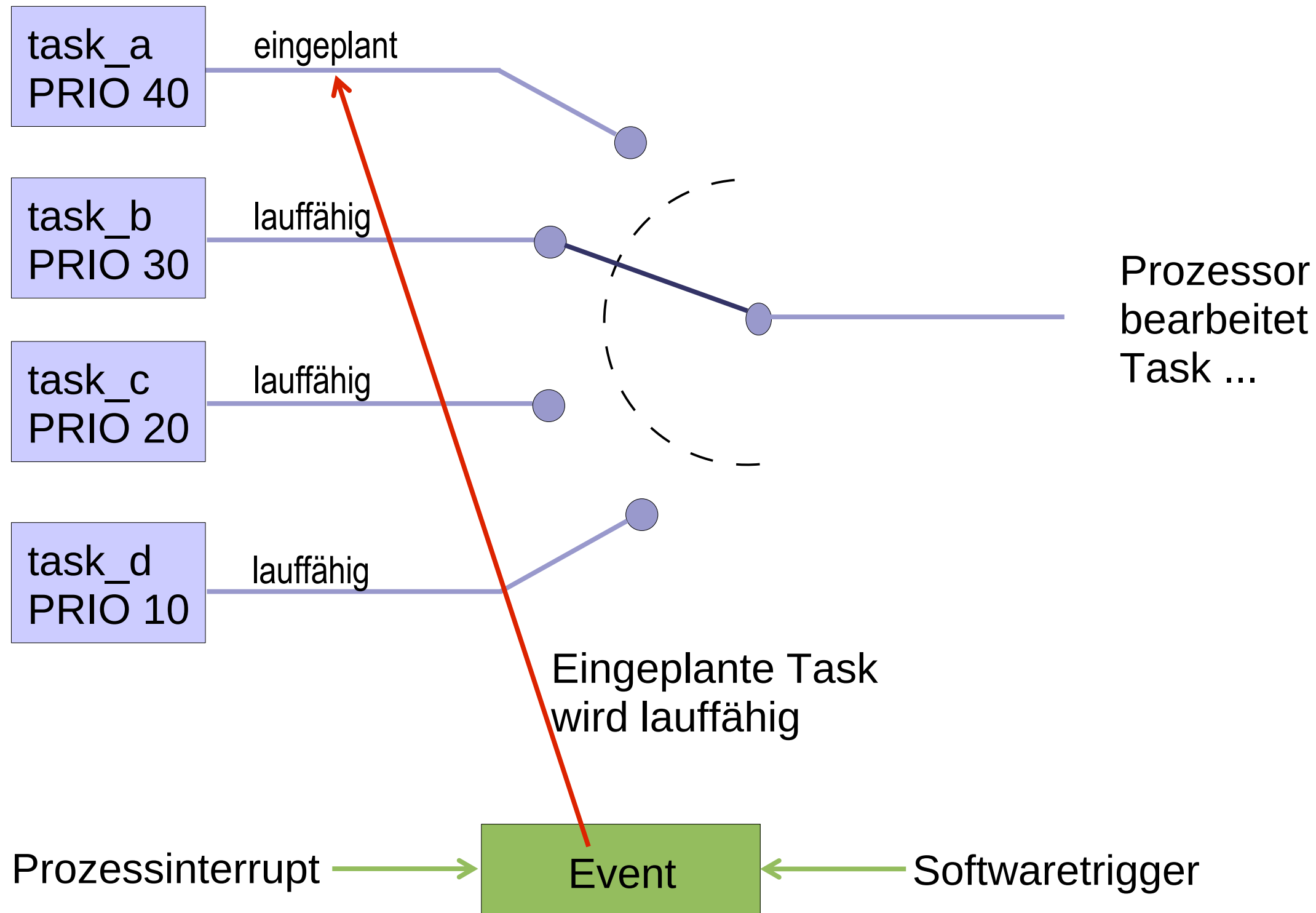
Ereigniseinplanung von Tasks: Prozessinterrupt



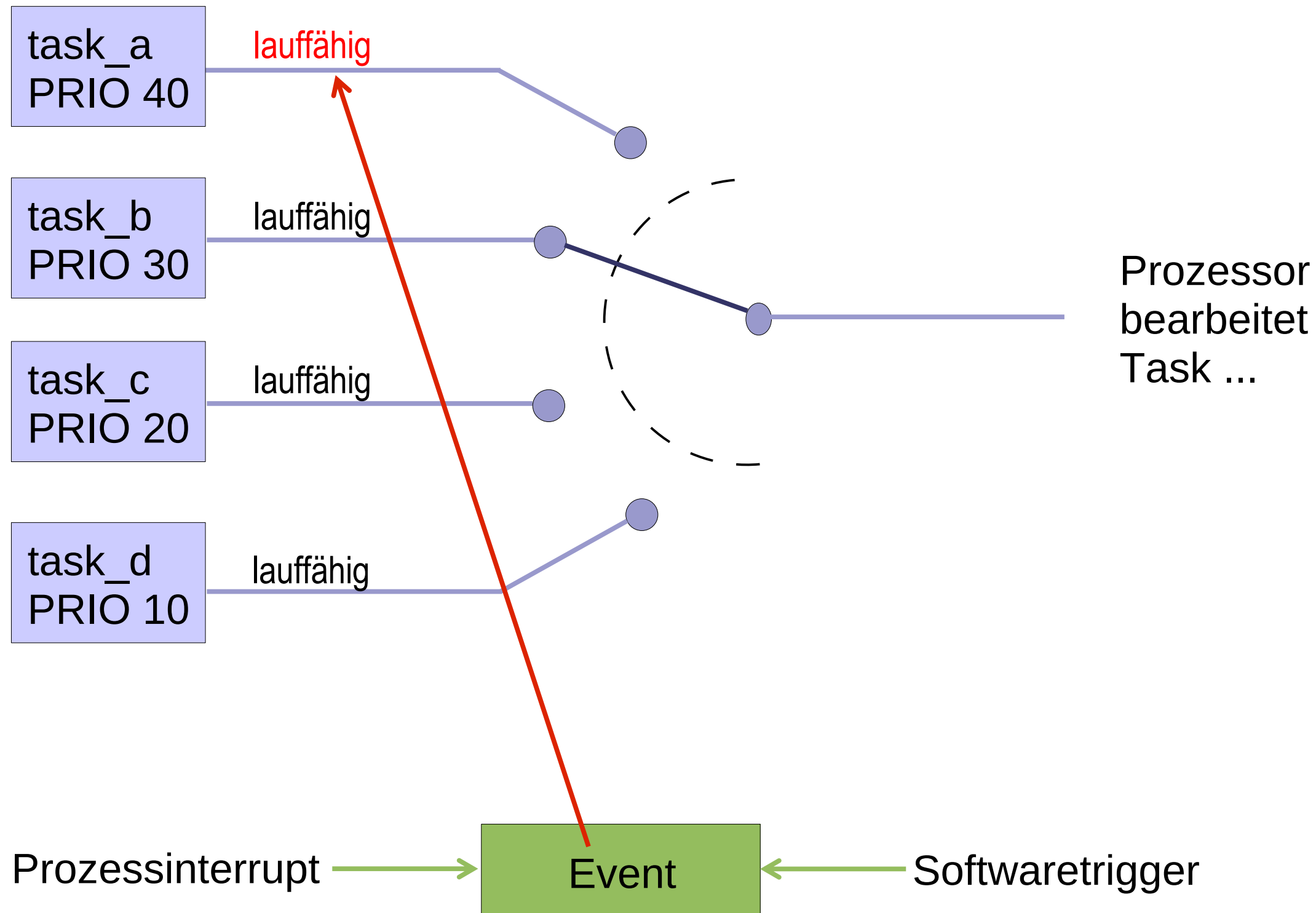
Ereigniseinplanung von Tasks: Prozessinterrupt



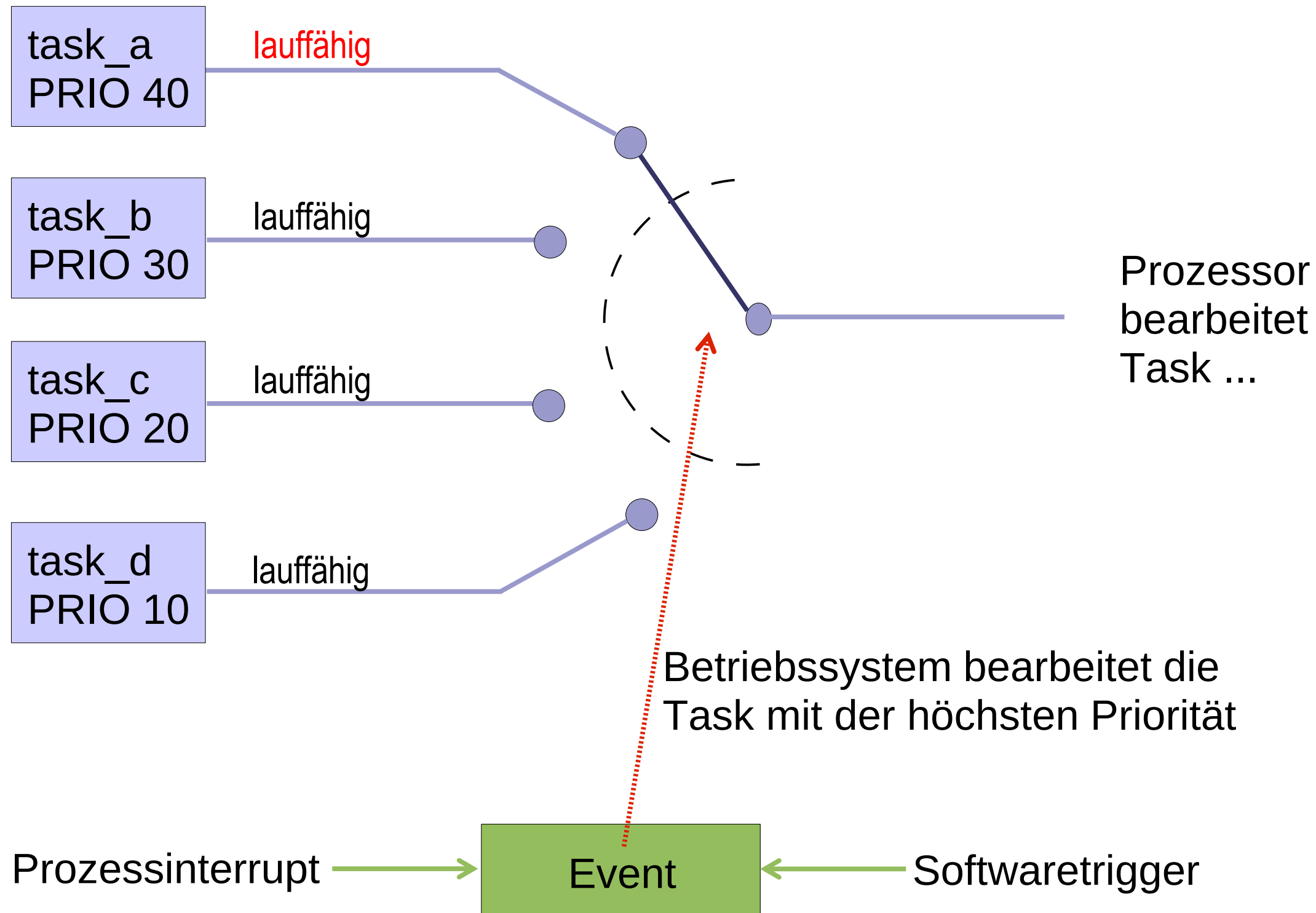
Ereigniseinplanung von Tasks: Prozessinterrupt



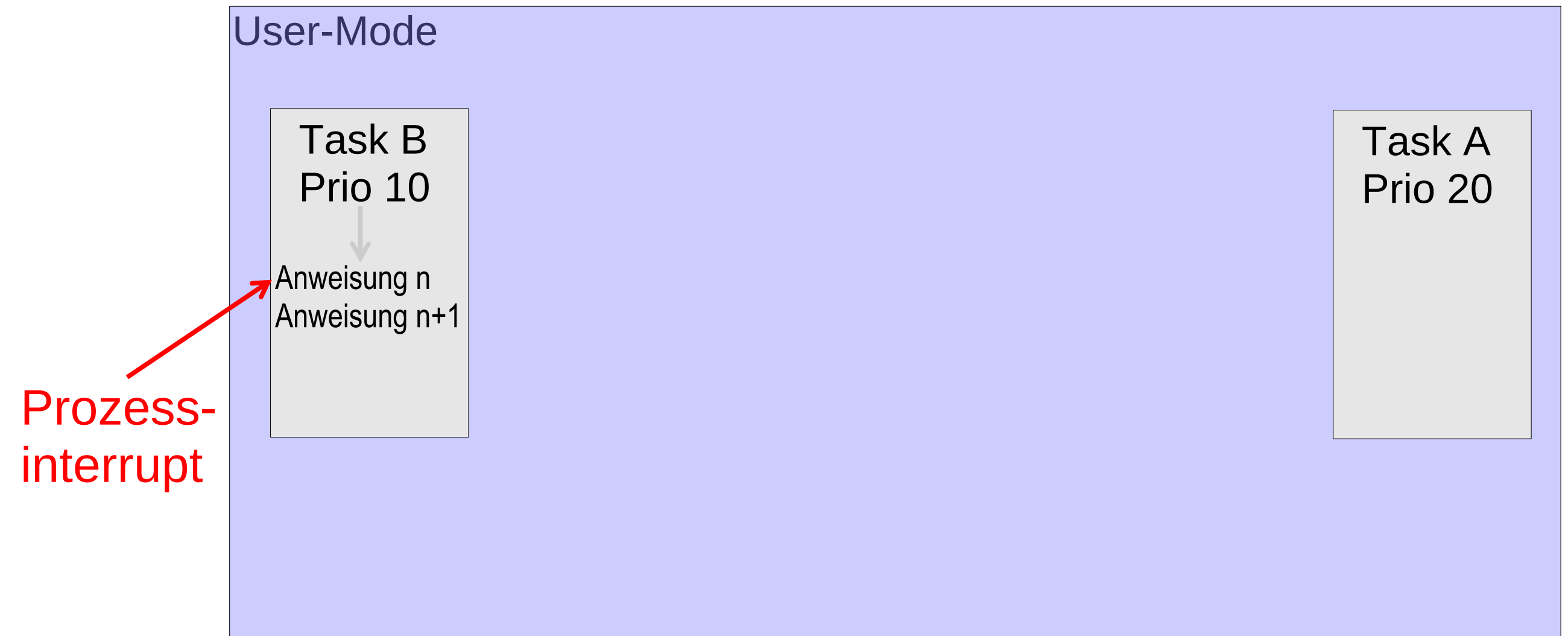
Ereigniseinplanung von Tasks: Prozessinterrupt



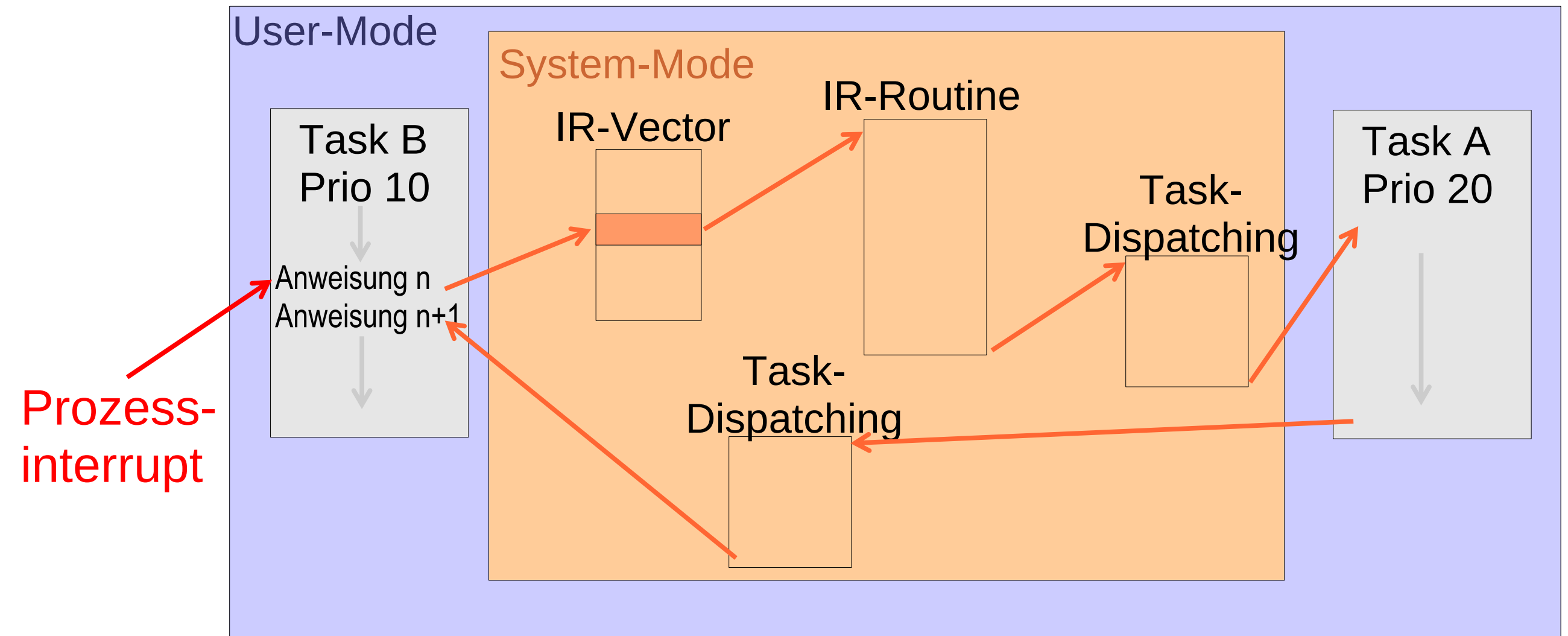
Ereigniseinplanung von Tasks: Prozessinterrupt



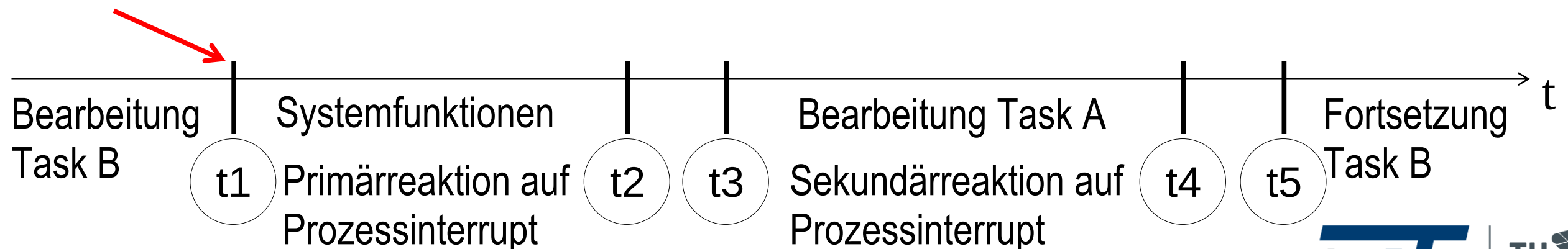
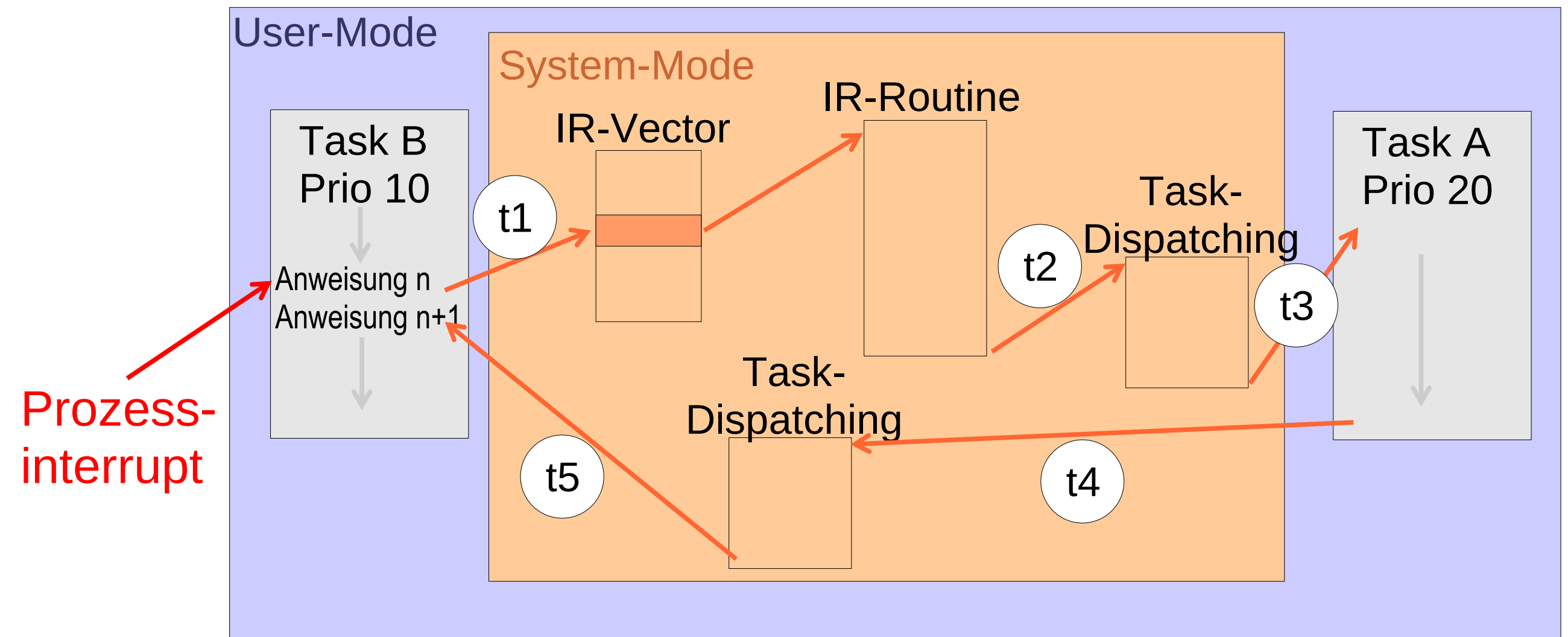
Ereigniseinplanung von Tasks: Prozessinterrupt



Ereigniseinplanung von Tasks: Prozessinterrupt

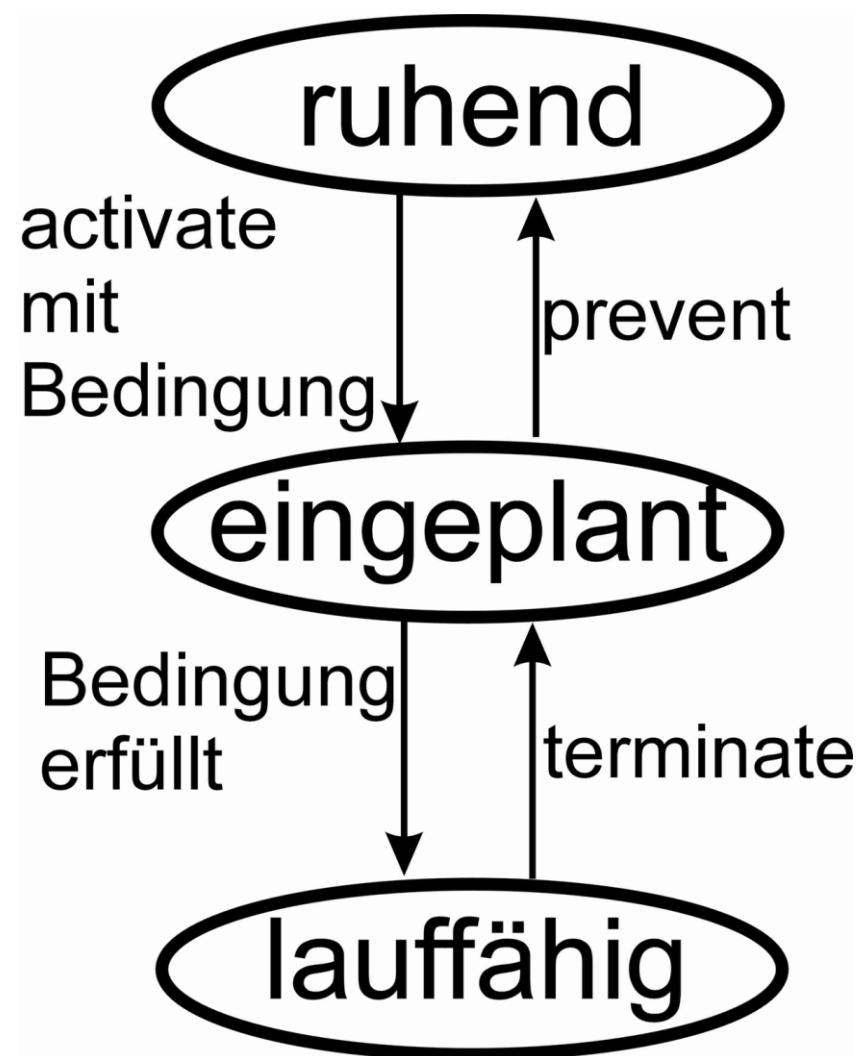


Ereigniseinplanung von Tasks: Prozessinterrupt



Ereigniseinplanung von Tasks: Prozessinterrupt

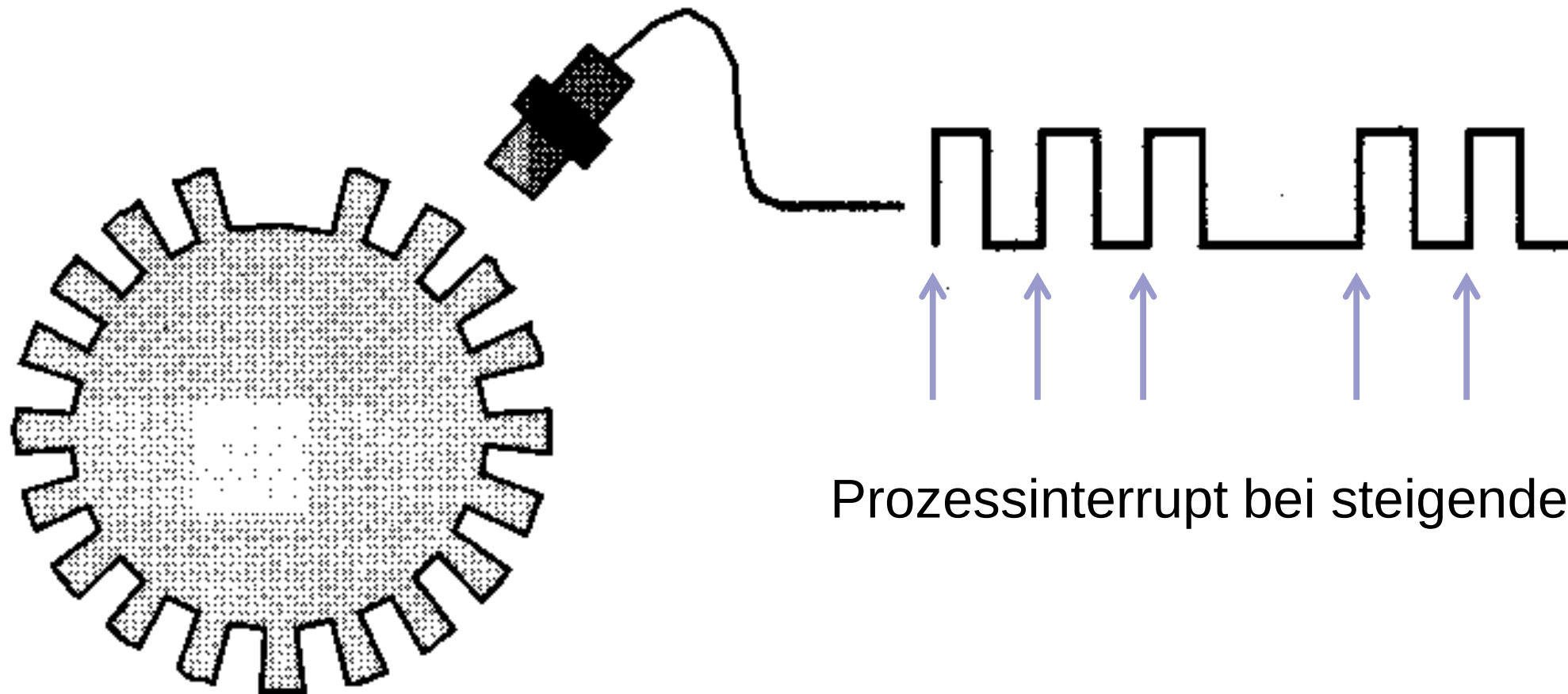
Ausschnitt aus Taskzustandsgraph für Einplanung



Beispiel für die Einplanung im Multitasking mit Prozessinterrupts

Ereigniseinplanung von Tasks: Prozessinterrupt

Beispiel: Impulse zählen und Impulsabstände messen



Achse mit variabler Geschwindigkeit

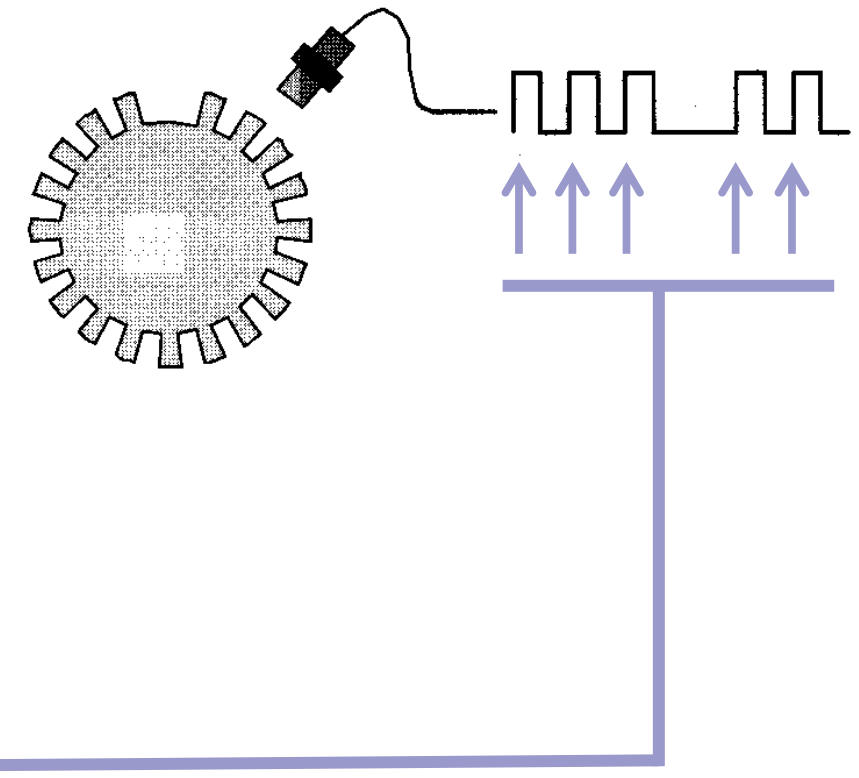
Ereigniseinplanung von Tasks: Prozessinterrupt

```
uint32_t count      = 0;
uint32_t anzahl_zahne = 0;
uint32_t state      = 1;
uint32_t time_now   = 0;
uint32_t time_old   = 0;
uint32_t diff_new   = 0;
uint32_t diff_old   = 0;
```

```
static TaskHandle_t zaehlen_Handle = NULL;
static TaskHandle_t init_anzahl_zahne_Handle = NULL;
static TaskHandle_t interrupt_target_Handle = NULL;
```

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
    vTaskResume(interrupt_target_Handle);
    return;
}
```

```
int main(void){
    ...
    TaskHandle_t Handle = NULL;
    xTaskCreate(init_anzahl_zahne, "", 128, NULL, 20, &init_anzahl_zahne_Handle);
    osKernelStart();
    while (1){}
}
```



Ereigniseinplanung von Tasks: Prozessinterrupt

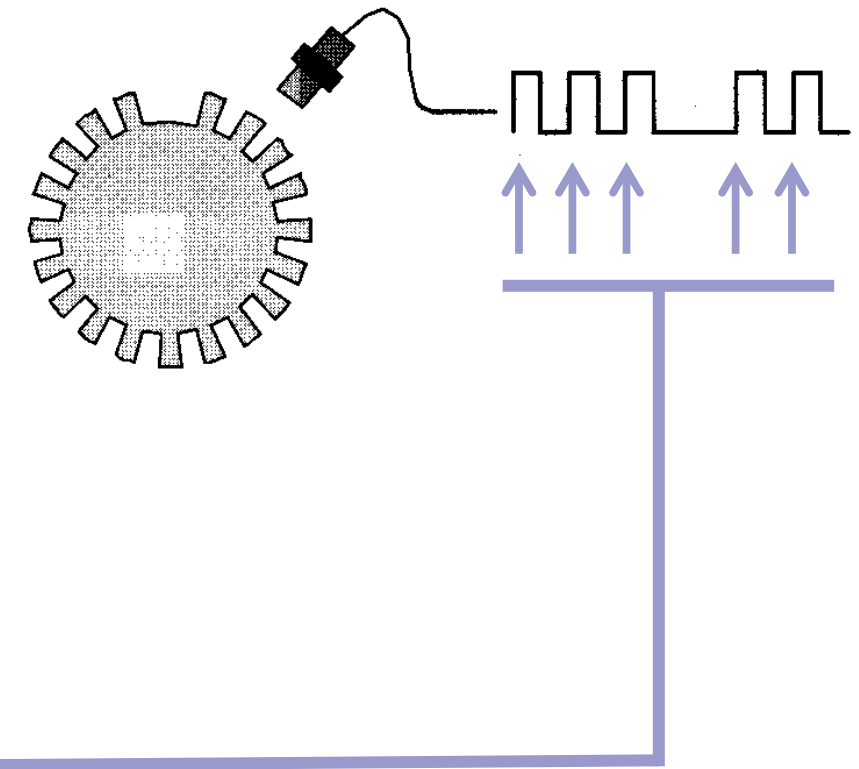
```
uint32_t count          = 0;
uint32_t anzahl_zahne   = 0;
uint32_t state          = 1;
uint32_t time_now       = 0;
uint32_t time_old       = 0;
uint32_t diff_new       = 0;
uint32_t diff_old       = 0;
```

Variablen für alle Tasks
zugänglich initialisieren

```
static TaskHandle_t zaehlen_Handle      = NULL;
static TaskHandle_t init_anzahl_zahne_Handle = NULL;
static TaskHandle_t interrupt_target_Handle = NULL;
```

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
    vTaskResume(interrupt_target_Handle);
    return;
}
```

```
int main(void){
    ...
    TaskHandle_t Handle = NULL;
    xTaskCreate(init_anzahl_zahne, "", 128, NULL, 20, &init_anzahl_zahne_Handle);
    osKernelStart();
    while (1){}
}
```



Ereigniseinplanung von Tasks: Prozessinterrupt

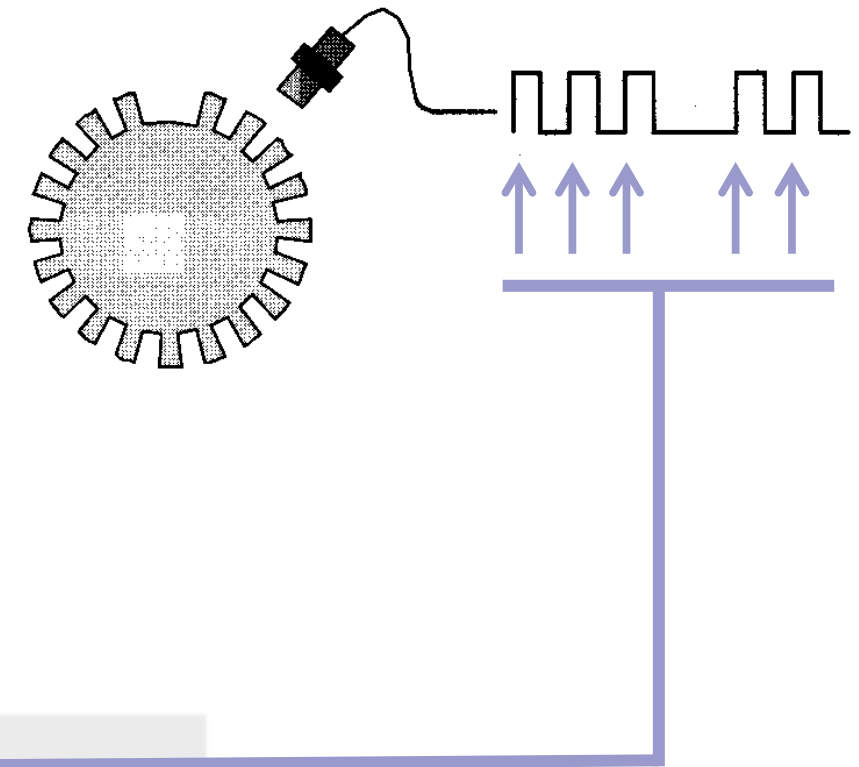
```
uint32_t count          = 0;
uint32_t anzahl_zahne   = 0;
uint32_t state          = 1;
uint32_t time_now       = 0;
uint32_t time_old       = 0;
uint32_t diff_new       = 0;
uint32_t diff_old       = 0;
```

```
static TaskHandle_t zaehlen_Handle = NULL;
static TaskHandle_t init_anzahl_zahne_Handle = NULL;
static TaskHandle_t interrupt_target_Handle = NULL;
```

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
    vTaskResume(interrupt_target_Handle);
    return;
}
```

Interrupt Ziel festlegen

```
int main(void){
    ...
    TaskHandle_t Handle = NULL;
    xTaskCreate(init_anzahl_zahne, "", 128, NULL, 20, &init_anzahl_zahne_Handle);
    osKernelStart();
    while (1){}
}
```



Ereigniseinplanung von Tasks: Prozessinterrupt

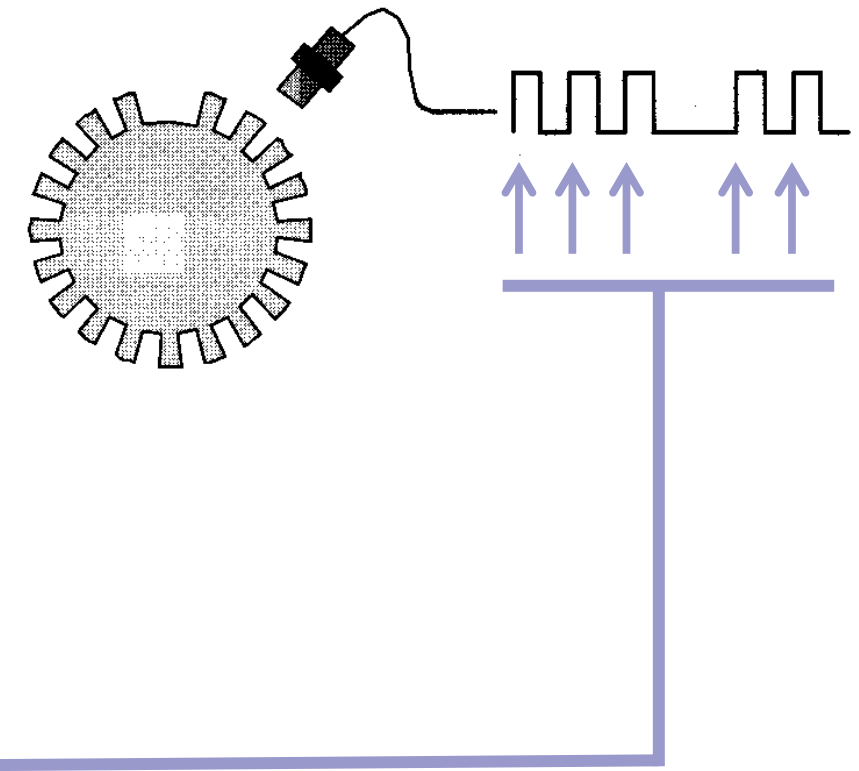
```
uint32_t count      = 0;
uint32_t anzahl_zahne = 0;
uint32_t state      = 1;
uint32_t time_now   = 0;
uint32_t time_old   = 0;
uint32_t diff_new   = 0;
uint32_t diff_old   = 0;
```

```
static TaskHandle_t zaehlen_Handle = NULL;
static TaskHandle_t init_anzahl_zahne_Handle = NULL;
static TaskHandle_t interrupt_target_Handle = NULL;
```

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
    vTaskResume(interrupt_target_Handle);
    return;
}
```

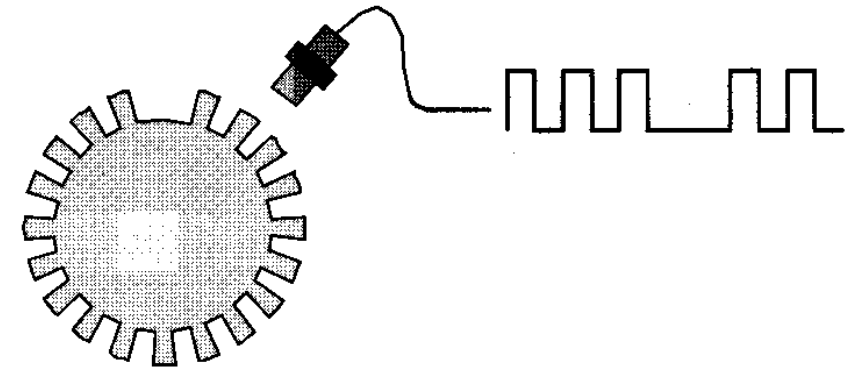
```
int main(void){
    ...
    TaskHandle_t Handle = NULL;
    xTaskCreate(init_anzahl_zahne, "", 128, NULL, 20, &init_anzahl_zahne_Handle);
    osKernelStart();
    while (1){}
}
```

Initialisierungstask starten



Ereigniseinplanung von Tasks: Prozessinterrupt

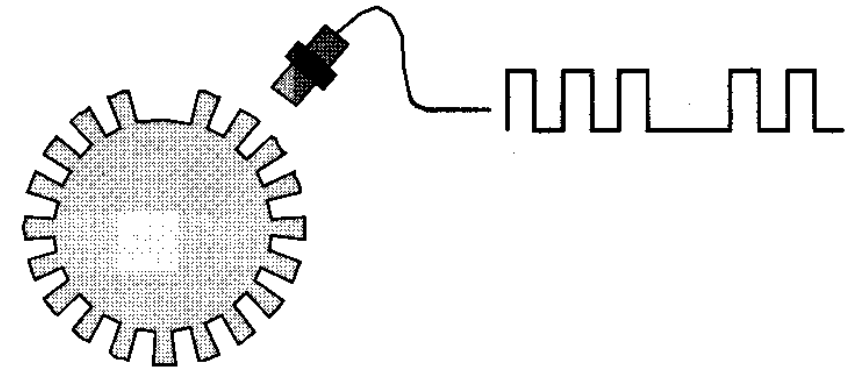
```
void init_anzahl_zahne(){
    for(;;){
        vTaskSuspend(NULL);
        time_now = (uint32_t) xTaskGetTickCount();
        diff_now = time_now - time_old;
        switch (state){
            case 1:
                diff_old = diff_new;
                state = 2;
                break;
            case 2:
                if((float) diff_now > (1.5 * (float)diff_old)){
                    count = 1, state = 3;
                }break;
            case 3:
                if((float)diff_now > (1.5 * (float)diff_old)){
                    anzahl_zahne = count;
                    count = 1;
                    xTaskCreate(zahlen, "", 128, NULL, 10, &zahlen_Handle);
                    interrupt_target_Handle = zahlen_Handle;
                    vTaskDelete(NULL);
                }else{
                    count = count + 1;
                }
            }
        }
        time_old = time_new; diff_old = diff_new;
    }
}
```



In ersten Durchlauf werden die Variablen nur initialisiert und der Zustand auf 2 gesetzt.

Ereigniseinplanung von Tasks: Prozessinterrupt

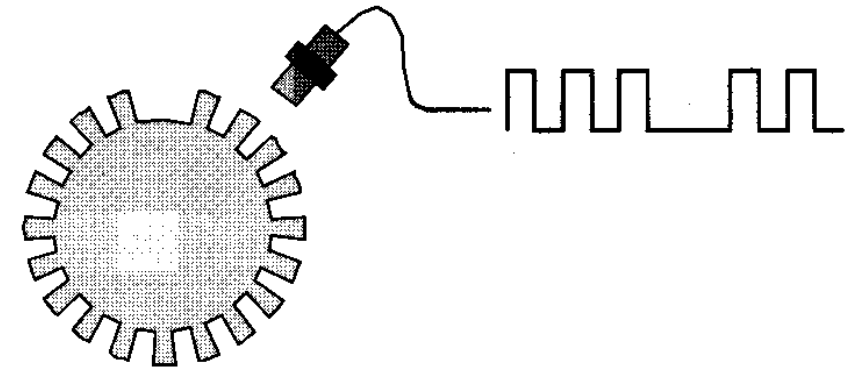
```
void init_anzahl_zaehe(){
    for(;;){
        vTaskSuspend(NULL);
        time_now = (uint32_t) xTaskGetTickCount();
        diff_now = time_now - time_old;
        switch (state){
            case 1:
                diff_old = diff_new;
                state = 2;
                break;
            case 2:
                if((float) diff_now > (1.5 * (float)diff_old)){
                    count = 1, state = 3;
                }break;
            case 3:
                if((float)diff_now > (1.5 * (float)diff_old)){
                    anzahl_zaehe = count;
                    count = 1;
                    xTaskCreate(zahlen, "", 128, NULL, 10, &zahlen_Handle);
                    interrupt_target_Handle = zahlen_Handle;
                    vTaskDelete(NULL);
                }else{
                    count = count + 1;
                }
            }
        }
        time_old = time_new; diff_old = diff_new;
    }
}
```



In Zustand 2 wird zuerst der Start (größere Lücke) gesucht und wenn diese gefunden wurde in Zustand 3 gewechselt.

Ereigniseinplanung von Tasks: Prozessinterrupt

```
void init_anzahl_zahne(){
    for(;;){
        vTaskSuspend(NULL);
        time_now = (uint32_t) xTaskGetTickCount();
        diff_now = time_now - time_old;
        switch (state){
            case 1:
                diff_old = diff_new;
                state = 2;
                break;
            case 2:
                if((float) diff_now > (1.5 * (float)diff_old)){
                    count = 1, state = 3;
                }break;
            case 3:
                if((float)diff_now > (1.5 * (float)diff_old)){
                    anzahl_zahne = count;
                    count = 1;
                    xTaskCreate(zahlen, "", 128, NULL, 10, &zahlen_Handle);
                    interrupt_target_Handle = zahlen_Handle;
                    vTaskDelete(NULL);
                }else{
                    count = count + 1;
                }
            }
        }
        time_old = time_new; diff_old = diff_new;
    }
}
```

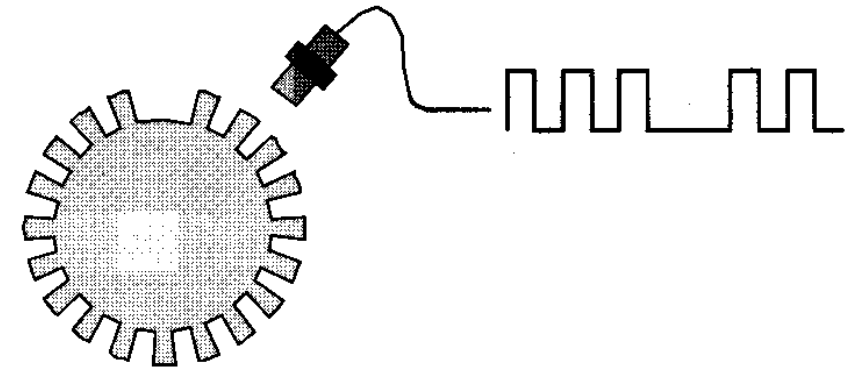


In Zustand 3 werden die Zähne gezählt, bis wieder der Start erreicht wurde.

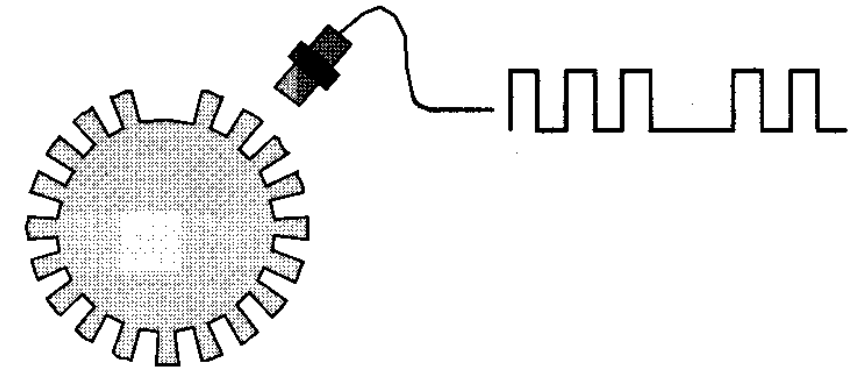
Ereigniseinplanung von Tasks: Prozessinterrupt

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
    vTaskResume(interrupt_target_Handle);
    return;
}
...
case 3:
    if((float)diff_now > (1.5 * (float)diff_old)){
        anzahl_zahne = count;    Variablen für Übergabe vorbereiten
        count = 1;
        xTaskCreate(zahlen, "", 128, NULL, 10, &zahlen_Handle);
        interrupt_target_Handle = zahlen_Handle;
        vTaskDelete(NULL);
    }
    ...

void zahlen(){
    for(;;){
        vTaskSuspend(NULL);
        count = count + 1;
        if(count >= anzahl_zahne){
            count = 0;
        }
    }
}
```



Ereigniseinplanung von Tasks: Prozessinterrupt

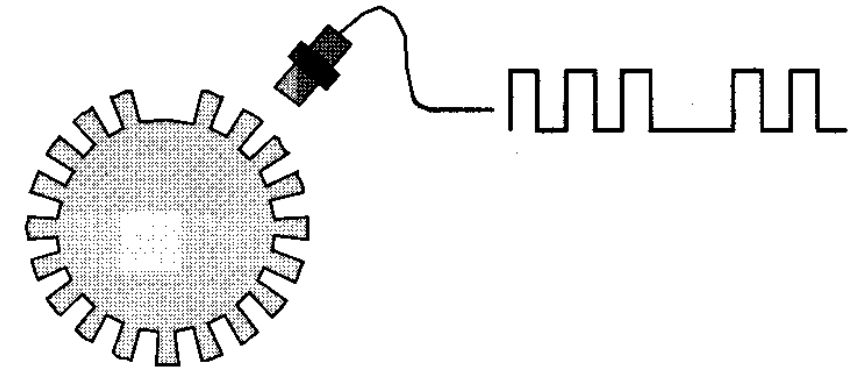


```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
    vTaskResume(interrupt_target_Handle);
    return;
}
...
case 3:
    if((float)diff_now > (1.5 * (float)diff_old)){
        anzahl_zahne = count;    Variablen für Übergabe vorbereiten
        count = 1;
        xTaskCreate(zahlen, "", 128, NULL, 10, &zahlen_Handle);
        interrupt_target_Handle = zahlen_Handle;
        vTaskDelete(NULL);
    }
    ...
```

Neue Task starten und neues Ziel für die ISR festlegen

```
void zahlen(){
    for(;;){
        vTaskSuspend(NULL);
        count = count + 1;
        if(count >= anzahl_zahne){
            count = 0;
        }
    }
}
```

Ereigniseinplanung von Tasks: Prozessinterrupt

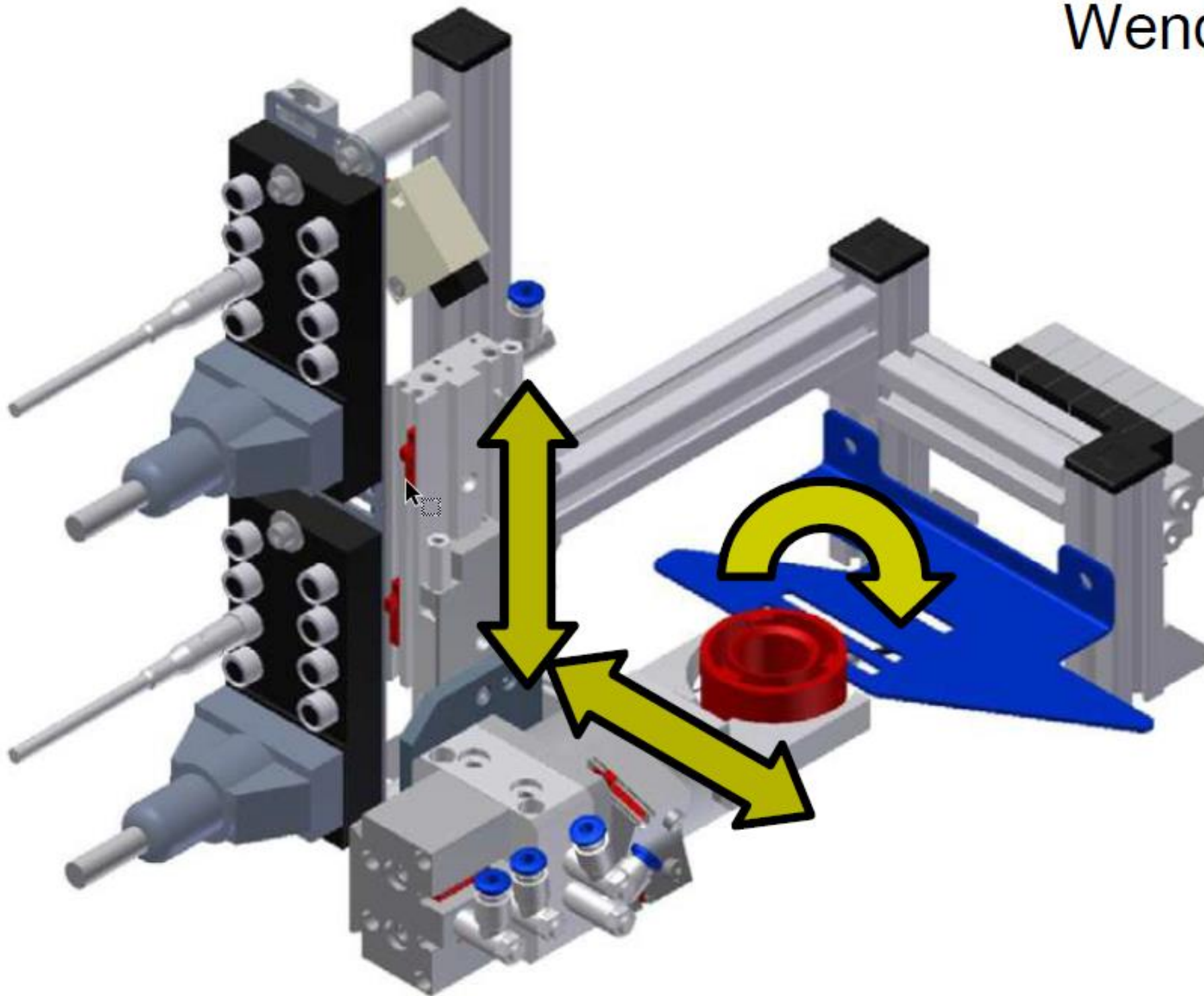


```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
    vTaskResume(interrupt_target_Handle);
    return;
}
...
case 3:
    if((float)diff_now > (1.5 * (float)diff_old)){
        anzahl_zahne = count;    Variablen für Übergabe vorbereiten
        count = 1;
        xTaskCreate(zahlen, "", 128, NULL, 10, &zahlen_Handle);
        interrupt_target_Handle = zahlen_Handle;
        vTaskDelete(NULL);    Task beendet sich selbst
    }
    ...

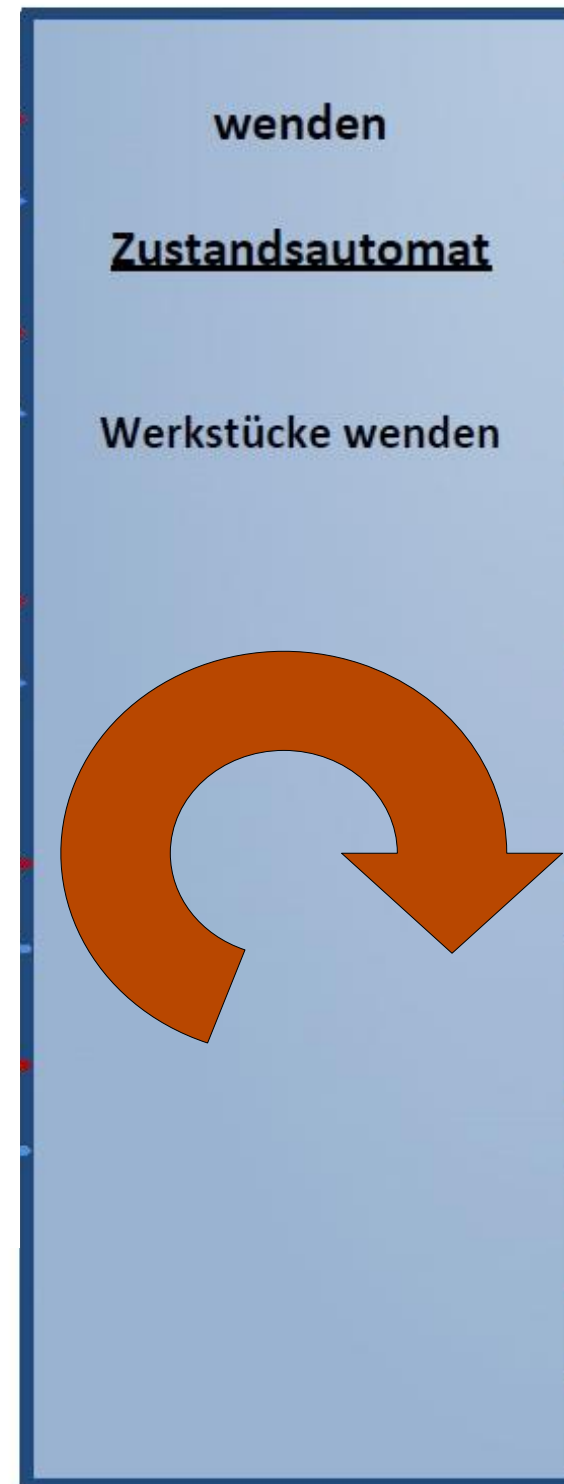
void zahlen(){
    for(;;){
        vTaskSuspend(NULL);
        count = count + 1;
        if(count >= anzahl_zahne){
            count = 0;
        }
    }
}
```

Ereigniseinplanung von Tasks

Wendeeinheit



Ereigniseinplanung von Tasks



Ereigniseinplanung von Tasks

